



Modul Praktikum

IoT

Internet of Things

Semester Ganjil
2026/2027

TIM PENYUSUN MODUL PRAKTIKUM

IoT Runtime and Integration System Laboratory 2026/2027

P304, Gedung Deli, Telkom University

Kepala Laboratorium	:	Dr. Junartho Halomoan, S.T., M.T	
Pembina Laboratorium	:	Istiqomah, S.T. M.Sc	
Koordinator Asisten	:	Bustan Nabiel Maulana	101022300013
Penanggung Jawab	:	Adrianus Wismar Nugrahanto	101022330019
Penyusun Modul	:		

- Immanuel Eben Haezer Aleteia 101022300172
- Muhammad Raditya Dwi Putra 101022440006
- Muhammad Apri Putra Pratama 101012500043
- Marvel Cristo Damanik 101012580014
- Rega Arzula Akbar 101022330038
- Wildan Khalid Wijaya 101022400126
- Ruifauzan Khiyar Wienarya 101022400038
- Mochammad Rifky Adiraysan 101022330158

ASISTEN PRAKTIKUM IRIS LABORATORY 2026/2027

No	NIM	Nama	Kode Asisten	Jabatan	Nomor HP	ID Line
1	101022300013	Bustan Nabiel Maulana	NAB	Koordinator Asisten	081216919968	tanmln
2	101022330019	Adrianus Wismar Nugrahanto	ADD	Wakil Koordinator Asisten	08811566526	shantika88
3	101052300029	Putri Natasya Villyadi	NAT	Sekretaris	082172772372	pnatas12
4	101052400100	Nathaniel Mildess	DES	Bendahara	087819956413	mildessnath_
5	101022300172	Immanuel Eben Haezer Joseph Aleteia	NUE	Kepala Divisi ASO	085829290950	eintswx
6	101022440006	Muhammad Raditya Dwi Putra	TWT	Anggota Divisi ASO	089676594781	muteless.seia
7	101012500043	Muhammad Apri Putra Pratama	MAP	Anggota Divisi ASO	082112774785	apriputra33
8	101012580014	Marvel Cristo Damanik	MCD	Anggota Divisi ASO	08999457440	marveldmk03
9	101022330038	Rega Arzula Akbar	RGZ	Kepala Divisi ARD	082126565892	arzula233
10	101022400126	Wildan Khalid Wijaya	WIL	Anggota Divisi ARD	082119761448	willdamnn
11	101022400038	Ruifauzan Khiyar Wienarya	ZAN	Anggota Divisi ARD	085793477750	ruienrya_
12	101022330158	Mochammad Rifky Adiraysan	IKY	Anggota Divisi ARD	0895384470333	m.rifky.a123
13	101022330079	Geraldi Ridwan Hakim	GRY	Kepala Divisi ADS	085224492559	geraldirh
14	101022330011	Agung Putra Pamungkas	GMW	Anggota Divisi ADS	082387359411	agugpp
15	101022300325	Agung Setia Dharma	OCU	Anggota Divisi ADS	081289458913	dharmapecl
16	101022300125	Serly Marleny	LYY	Kepala Divisi ADC	081388790318	serlymrll
17	101022330033	Monica Rizkia Nanda	ICA	Anggota Divisi ADC	082130661485	Ichaca25
18	101022400093	Moh. Fathirochman Yasin	FAT	Anggota Divisi ADC	085240181824	ftrysn
19	101022300129	Muhammad Raid Ihsan Akbar	BAR	Kepala Divisi AER	082124072608	mr.ihsanakbar
20	101022300195	Bryan Gabriel Izaac Sasabone	YEN	Anggota Divisi AER	082197881252	bryanssbne
21	101022400216	Rido Alip Fianto	MCU	Anggota Divisi AER	085779076891	rido237206219w
22	101022330034	Graceyana Ardhia Shinta Thalia Kottama	XIN	Anggota Divisi AER	081233477989	bee__thalia
23	101052300102	Luqmanul Hakim	LUV	Kepala Divisi ARM	082238264823	luqman0765
24	101012330221	Darma Al Gani	GAN	Anggota Divisi ARM	089513829923	darmapamatan
25	101022330157	Tirta Razzan	TRA	Anggota Divisi ARM	082257298778	tirta.razzan

LEMBAR REVISI

Yang bertanda tangan di bawah ini:

Nama	:	Istiqomah, S.T. M.Sc
NIP	:	25910022
Jabatan	:	Kepala Lab IoT Runtime and Integration System Laboratory

Dengan ini menyatakan pelaksanaan Revisi Modul Pemrograman Perangkat IoT untuk Prodi S1 Teknik Elektro, telah dilaksanakan dengan penjelasan sebagai berikut:

No	Modul	Keterangan Revisi	Tanggal Revisi Akhir
1	Modul 1	Penambahan materi mengenai Bluetooth Low Energy (BLE) beserta implementasinya	31 Agustus 2026
2	Modul 2	Pergantian topik materi dari Google Firebase dan Kodular menjadi server lokal menggunakan HTTP dengan komunikasi WebSocket	31 Agustus 2026
3	Modul 3	Perubahan penggunaan broker MQTT yang sebelumnya menggunakan AdafruitIO dengan broker lokal Mosquitto MQTT	31 Agustus 2026
4	Modul 4	Penambahan materi mengenai LoRaWAN beserta implementasinya menggunakan platform ANTARES	31 Agustus 2026
5	Modul 1, 2, 3, dan 4	Merapihkan format penulisan	31 Agustus 2026

LEMBAR PERNYATAAN

Yang bertanda tangan di bawah ini:

Nama	:	Istiqomah, S.T. M.Sc
NIP	:	25910022
Jabatan	:	Kepala Lab IoT Runtime and Integration System Laboratory

Menerangkan dengan sesungguhnya bahwa modul praktikum ini telah di-review dan akan digunakan untuk pelaksanaan praktikum di Semester Ganjil Tahun Akademik 2026/2027 di Laboratorium Electronics and Industrial Automation Fakultas Teknik Elektro Universitas Telkom.

Bandung, 31 Agustus 2026

Kepala Lab IoT Runtime and Integration System Laboratory	Koordinator Asisten IoT Runtime and Integration System Laboratory
Istiqomah, S.T. M.Sc	**Bustan Nabiel Maulana**
NIP 25910022	NIM 101022300013

VISI DAN MISI
FAKULTAS TEKNIK ELEKTRO

VISI:

Menjadi fakultas berstandar internasional yang berperan aktif dalam pengembangan pendidikan, riset, dan entrepreneurship di bidang Teknik Elektro, berbasis teknologi informasi.

MISI:

1. Menyelenggarakan sistem pendidikan yang berstandar internasional di bidang Teknik Elektro berbasis teknologi informasi.
2. Menyelenggarakan, menyebarluaskan, dan memanfaatkan hasil-hasil riset berstandar internasional di bidang Teknik Elektro.
3. Menyelenggarakan program entrepreneurship berbasis teknologi bidang Teknik Elektro di kalangan civitas akademika untuk mendukung pembangunan ekonomi nasional.
4. Mengembangkan jejaring dengan perguruan tinggi dan industri terkemuka dalam dan luar negeri dalam rangka kerja sama pendidikan, riset, dan entrepreneurship.
5. Mengembangkan sumber daya untuk mencapai keunggulan dalam pendidikan, riset, dan entrepreneurship.

VISI DAN MISI
PROGRAM STUDI S1 TEKNIK ELEKTRO

VISI:

Menjadi program studi berstandar internasional yang berperan aktif dalam pengembangan pendidikan, riset, dan kewirausahaan serta menghasilkan sarjana di bidang Sistem Elektronika, Sistem Kendali, atau Sistem Tertanam.

MISI:

1. Menyelenggarakan pendidikan berstandar internasional untuk menghasilkan lulusan yang menguasai ilmu pengetahuan dan teknologi Sistem Elektronika, Sistem Kendali atau Sistem Tertanam.
2. Mengembangkan dan menyebarluaskan ilmu pengetahuan dan teknologi Sistem Elektronika, Sistem Kendali, dan Sistem Tertanam yang diakui secara internasional dengan melibatkan mahasiswa secara aktif.
3. Memanfaatkan ilmu pengetahuan dan teknologi Sistem Elektronika, Sistem Kendali, dan Sistem Tertanam untuk pengembangan kewirausahaan di kalangan sivitas akademika dalam rangka mendukung pembangunan ekonomi nasional.
4. Mengembangkan jejaring dengan perguruan tinggi dan industri terkemuka dalam dan luar negeri dalam rangka kerja sama pendidikan, riset, dan kewirausahaan dalam bidang Sistem Elektronika, Sistem Kendali, atau Sistem Tertanam.
5. Mengembangkan sumber daya untuk mencapai keunggulan dalam pendidikan, riset, dan kewirausahaan dalam bidang Sistem Elektronika, Sistem Kendali, atau Sistem Tertanam.

ATURAN PRAKTIKUM LABORATORIUM FAKULTAS TEKNIK ELEKTRO TELKOM UNIVERSITY

Untuk memastikan bahwa setiap sesi praktikum berlangsung dengan lancar, ada beberapa aturan dan pedoman yang perlu diperhatikan oleh setiap praktikan. Kepatuhan terhadap aturan ini tidak hanya akan membantu menciptakan lingkungan yang kondusif dan aman, tetapi juga akan memastikan bahwa setiap praktikan dapat memaksimalkan kegiatan praktikum. Berikut adalah beberapa hal yang harus diperhatikan dan dipatuhi oleh setiap praktikan.

Praktikan yang Tidak Diizinkan Masuk

Praktikan tidak akan diizinkan masuk atau mengikuti praktikum jika :

- Terlambat maksimal 20 menit dari dimulainya suatu shift.

  Shift 1 : 06.30 – 09.00 WIB
  Shift 2 : 09.30 – 12.00 WIB
  Shift 3 : 12.30 – 15.00 WIB
  Shift 4 : 15.30 – 18.00 WIB

- Tidak terdaftar di Seelabs.
- Selain itu, alasan seperti pakaian tetap diperbolehkan mengikuti praktikum dengan sanksi yang telah diberikan sesuai ketetapan laboratorium yang bersangkutan.

Pengajuan Praktikum Susulan

Apabila tidak mengikuti praktikum, maka praktikan dapat mengajukan praktikum susulan dengan ketetapan yang dibuat oleh laboran, yaitu :

- Susulan dilakukan setelah praktikum selesai di waktu yang telah ditetapkan oleh laboran.
- Hanya praktikan yang disetujui oleh laboran yang diperbolehkan susulan (Sakit dengan bukti surat sakit dokter, rawat inap, kematian keluarga inti, dan dispensasi dari BK).
- Pengajuan susulan dilakukan praktikan melalui OA Seelabs (@jit0659i) dengan melampirkan bukti.

PANDUAN UMUM KESELAMATAN DAN PENGGUNAAN PERALATAN LABORATORIUM

Keselamatan

Untuk mewujudkan praktikum yang aman diperlukan partisipasi seluruh praktikan dan asisten pada kegiatan praktikum yang bersangkutan. Dengan demikian, kepatuhan setiap praktikan terhadap uraian panduan pada bagian ini akan sangat membantu mewujudkan praktikum yang aman.

Bahaya Listrik

Perhatikan dan pelajari tempat-tempat sumber listrik (stop-kontak dan circuit breaker) serta cara menyala-matikannya. Jika melihat ada kerusakan yang dapat menimbulkan bahaya, segera laporkan pada asisten.

- Hindari daerah atau benda yang berpotensi menimbulkan bahaya listrik (sengatan listrik/ strum) secara tidak disengaja, misalnya kabel jala-jala yang terkelupas, dll.
- Tidak melakukan sesuatu yang dapat menimbulkan bahaya listrik pada diri sendiri atau orang lain.
- Keringkan bagian tubuh yang basah karena, misalnya, keringat atau sisa air wudhu.
- Selalu waspada terhadap bahaya listrik pada setiap aktivitas praktikum.

Kecelakaan akibat bahaya listrik yang sering terjadi adalah tersengat arus listrik. Berikut ini adalah hal-hal yang harus dilakukan oleh praktikan jika hal tersebut terjadi :

- Jangan panik;
- Matikan semua peralatan elektronik dan sumber listrik di meja masing-masing dan di meja orang yang tersengat arus listrik;
- Bantu orang yang tersengat arus listrik untuk melepaskan diri dari sumber listrik; dan
- Beritahukan dan minta bantuan pada asisten, praktikan lain, serta orang di sekitar tentang terjadinya kecelakaan akibat bahaya listrik.

Bahaya Api atau Panas Berlebih

Dilarang membawa benda-benda yang mudah terbakar (korek api, gas, dll.) ke dalam ruangan praktikum bila tidak diisyaratkan dalam modul praktikum.

- Jangan melakukan sesuatu yang dapat menimbulkan api, percikan api atau panas yang berlebihan.
- Jangan melakukan sesuatu yang dapat menimbulkan bahaya api atau panas berlebih pada diri sendiri dan orang lain.

- Selalu waspada terhadap bahaya api atau panas berlebih pada setiap aktivitas praktikum.

Berikut ini adalah hal-hal yang harus diikuti praktikan jika menghadapi bahaya api atau panas berlebih :

- Jangan panik;
- Beritahukan dan minta bantuan pada asisten, praktikan lain, serta orang di sekitar tentang terjadinya bahaya api atau panas berlebih;
- Matikan semua peralatan elektronik dan sumber listrik ruangan praktikum;
- Menjauh dari ruang praktikum

Bahaya Lain

Untuk menghindari terjadinya hal-hal yang tidak diinginkan selama pelaksanaan praktikum, perhatikan juga hal-hal berikut:

- Jangan membawa benda tajam (pisau, gunting, dan sejenisnya) ke ruangan praktikum bila tidak diperlukan untuk pelaksanaan percobaan.
- Jangan memakai perhiasan dari logam, misalnya cincin, kalung, gelang, dll.
- Hindari daerah, benda, atau logam yang memiliki bagian tajam dan dapat melukai.
- Hindari melakukan sesuatu yang dapat menimbulkan luka pada diri sendiri atau orang lain, seperti bermain-main saat praktikum berlangsung.

DENAH EVAKUASI
LANTAI 3 - GEDUNG DELI/P





DAFTAR ISI

TIM PENYUSUN MODUL PRAKTIKUM i
ASISTEN PRAKTIKUM IRIS LABORATORY 2026/2027 ii
LEMBAR REVISI iii
LEMBAR PERNYATAAN iv
VISI DAN MISI FAKULTAS TEKNIK ELEKTRO v
VISI DAN MISI PROGRAM STUDI S1 TEKNIK ELEKTRO vi
ATURAN PRAKTIKUM LABORATORIUM FAKULTAS TEKNIK ELEKTRO TELKOM UNIVERSITY vii
PANDUAN UMUM KESELAMATAN DAN PENGGUNAAN PERALATAN LABORATORIUM viii
DENAH EVAKUASI LANTAI 3 - GEDUNG DELI/P x
DAFTAR ISI xi
DAFTAR GAMBAR xiii
INTRODUCTION TO INTERNET OF THINGS (IOT) 1
Apa itu Internet of Things (IoT)? 1
Architecture Layer pada IoT 1
Physical Layer/Perception Layer 1
Network Layer 1
Application Layer 4
MODUL 1: KONEKTIVITAS BLUETOOTH 5
1.1 Bluetooth 6
1.2 Jenis-Jenis Bluetooth 7
1.2.1 Bluetooth 1.0 7
1.2.2 Bluetooth 2.0 7
1.2.3 Bluetooth 3.0 7
1.2.4 Bluetooth 4.0 (Bluetooth Low Energy) 7
1.2.5 Bluetooth 5.0 8
1.2.6 Bluetooth 6.0 8
1.3 Alur Komunikasi Bluetooth 8
1.3.1 Bluetooth Serial 8
1.3.2 Bluetooth Low Energy 9
1.4 Protokol Bluetooth 10
1.5 Implementasi peer-to-peer Bluetooth 12
1.5.1 Menggunakan Bluetooth Serial 12
1.5.2 Menggunakan Bluetooth Low Energy (BLE) 15
MODUL 2: KOMUNIKASI WEBSOCKET DENGAN SERVER LOKAL 21
2.1 Hypertext Transfer Protocol (HTTP) 22
2.1.1 CRUD 22
2.1.2 Metode Komunikasi Protokol HTTP 22
2.2 WebSocket 24
2.3 Implementasi Protokol WebSocket 25
2.3.1 Server Python Raspberry Pi 25
2.3.2 Source Code ESP32 27
MODUL 3: KONEKTIVITAS MQTT DENGAN MICROPYTHON, RASPBERRY PI PICO W, DAN ECLIPSE MOSQUITTO 31
3.1 MQTT 32
3.2 Kelebihan MQTT 32

3.3 Alur Komunikasi MQTT 32
3.3.1 Publish-Subscribe 32
3.3.2 Topics 33
3.3.3 Broker 33
3.4 Eclipse Mosquitto 33
3.5 MicroPython 34
3.6 Raspberry Pi Pico W 35
3.7 Implementasi Protokol MQTT 35
3.7.1 Setup Broker Lokal dengan Raspberry Pi 35
3.7.2 Dengan Autentikasi 36
3.7.3 Setup Library MQTT pada Raspberry Pi Pico W 37
3.7.4 Source Code Raspberry Pi Pico W 38
MODUL 4: KOMUNIKASI LONG RANGE (LoRa) 42
4.1 LoRa 43
4.2 Alur Komunikasi LoRa 43
4.3 LoRaWAN 44
4.4 Tipe Kelas LoRaWAN 45
4.4.1 Class A 45
4.4.2 Class B 45
4.4.3 Class C 46
4.5 ANTARES 46
4.6 Implementasi Protokol LoRa 46
4.6.1 Peer-to-peer LoRa 46
4.6.2 Implementasi LoRaWAN Menggunakan ANTARES 50
LAMPIRAN 59
LAMPIRAN PINOUT DIAGRAM MIKROKONTROLER 60
ESP32 DEVKIT V1 - DOIT 60
ESP32-S3 60
RASPBERRY PI PICO W 61
COSMIC LORA AURORA V2 62
LAMPIRAN MODUL 1 63
Source Code Bluetooth Master 63
Source Code Bluetooth Slave 63
Source Code BLE Central/Client 64
Source Code BLE Peripheral/Server 66
LAMPIRAN MODUL 2 69
WebSocket Python Server Source Code 69
Source Code WebSocket dengan ESP32 69
LAMPIRAN MODUL 3 72
Source Code untuk Publish MQTT ke Mosquitto MQTT 72
Source Code untuk Subscribe MQTT dari Mosquitto MQTT 73
LAMPIRAN MODUL 4 75
Source Code untuk Transmitter LoRa 75
Source Code untuk Receiver LoRa 76
LoRaWAN Source Code 77

DAFTAR GAMBAR

Gambar 1.1 Logo Bluetooth 6
Gambar 1.2 Jacobus Cornelis Haartsen 6
Gambar 1.3 Alur Komunikasi Bluetooth Serial 8
Gambar 1.4 Alur Komunikasi Bluetooth Low Energy 9
Gambar 1.5 Struktur GATT 10
Gambar 1.6 Visual P2P 12
Gambar 1.7 Output pada Serial Monitor 15
Gambar 2.1 Visualisasi HTTP 22
Gambar 2.2 Metode Komunikasi HTTP 23
Gambar 2.3 Alur Komunikasi WebSocket 24
Gambar 2.4 Output pada Server WebSocket 30
Gambar 3.1 Visualisasi Publish - Subscribe 32
Gambar 3.2 Visualisasi Broker 33
Gambar 3.3 Logo Mosquitto MQTT 33
Gambar 3.4 Implementasi MQTT dengan Raspberry Pi Pico W 35
Gambar 3.5 Tools > Manage Packages 38
Gambar 4.1 LoRa Bandwidth - Range 43
Gambar 4.2 Arsitektur LoRaWAN 44
Gambar 4.3 Komunikasi LoRa Class A 45
Gambar 4.4 Komunikasi LoRa Class B 45
Gambar 4.5 Komunikasi LoRa Class C 46
Gambar 4.6 ANTARES Logo 46
Gambar 4.7 Visualisasi P2P LoRa 46
Gambar 4.8 Pendaftaran ANTARES 51
Gambar 4.9 Dashboard Application ANTARES 51
Gambar 4.10 Dialog Pembuatan Application ANTARES 51
Gambar 4.11 Halaman Device pada Application ANTARES 52
Gambar 4.12 Dialog Pembuatan Device ANTARES 52
Gambar 4.13 Detail Device ANTARES 52
Gambar 4.14 Detail Device ANTARES 53
Gambar 4.15 Parameter LoRaWAN Device ANTARES 53
Lampiran 1 Pinout ESP32 DEVKIT V1 - DOIT 60
Lampiran 2 Pinout ESP32-S3 60
Lampiran 3 Pinout RASPBERRY PI PICO W 61
Lampiran 4 Pinout Cosmic LoRa Aurora V2 62

INTRODUCTION TO INTERNET OF THINGS (IOT)

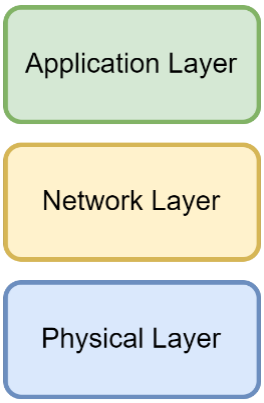
Apa itu Internet of Things (IoT)?

Internet of Things (IoT) adalah konsep yang menghubungkan objek fisik, sensor, aktuator, perangkat komputasi, dan aplikasi melalui jaringan agar perangkat dapat mengumpulkan, bertukar, serta memanfaatkan data. Komunikasi tersebut dapat berlangsung secara lokal maupun melalui internet dengan menggunakan protokol yang sesuai dengan kebutuhan sistem.

IoT dapat membantu berbagai aktivitas sehari-hari. Sebagai contoh, pengguna dapat mengendalikan peralatan rumah melalui aplikasi Android, memantau kondisi dan keamanan rumah dari jarak jauh, serta mengatur pemberian pakan hewan secara otomatis. Penerapan tersebut tetap memerlukan perancangan keamanan, keandalan jaringan, dan perlindungan data pengguna.

Architecture Layer pada IoT

Arsitektur IoT dapat dijelaskan menggunakan beberapa model. Salah satu model sederhana terdiri atas tiga lapisan, yaitu Physical/Perception, Network, dan Application, seperti yang ditunjukkan pada Gambar Arsitektur Layer.



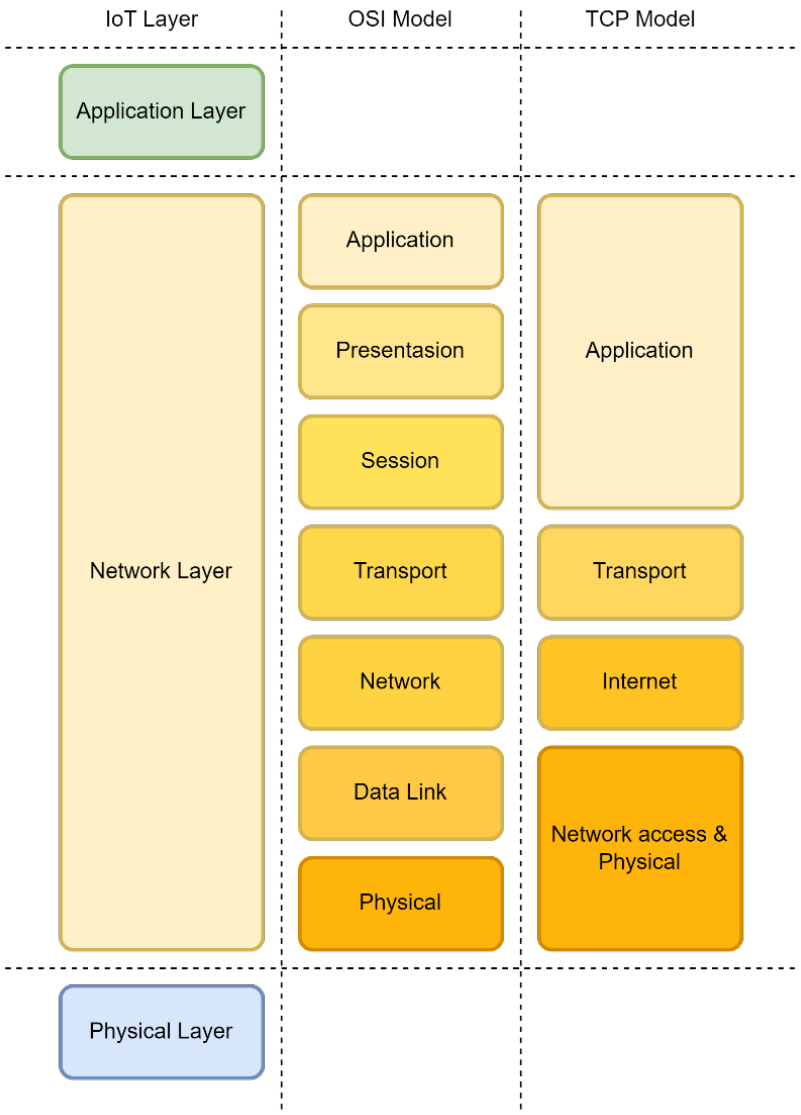
Gambar 1: Architecture layer pada IoT

Physical Layer/Perception Layer

Physical Layer atau Perception Layer merupakan lapisan yang berhubungan langsung dengan objek fisik. Lapisan ini mencakup sensor untuk melakukan akuisisi data, aktuator untuk menjalankan tindakan, serta perangkat kendali. Sebagai contoh, pada sistem rumah pintar, sensor dapat mengukur suhu atau mendeteksi kondisi keamanan, sedangkan aktuator dapat mengunci pintu, menyalakan lampu, atau mengatur suhu AC.

Network Layer

Dalam sistem IoT, Network Layer menyediakan konektivitas dan mekanisme pertukaran data antara perangkat pada Physical/Perception Layer dan layanan pada Application Layer. Komunikasi dapat berlangsung melalui jaringan lokal, gateway, internet, atau kombinasi ketiganya. Dua model referensi yang umum digunakan untuk menjelaskan komunikasi jaringan adalah model OSI dan model TCP/IP, sebagaimana ditunjukkan pada Perbandingan layer pada IoT, OSI, dan TCP/IP.



Gambar 2: Perbandingan layer pada IoT, OSI, dan TCP

OSI Model

a. **Physical**

Physical layer adalah layer paling bawah pada model OSI. Layer ini menentukan sifat fisik dari media yang digunakan untuk mengirimkan data antarperangkat. Contohnya, tingkat tegangan, jarak transmisi maksimum, konektor fisik, dan sebagainya. Pada layer ini, bit digital diubah menjadi sinyal listrik untuk koneksi kabel dan menjadi sinyal radio untuk transmisi nirkabel.

b. **Data Link**

Layer ini memfasilitasi komunikasi dan transfer data antarnode (contohnya, PC ke switch, switch ke router, dan router ke router). Alamat fisik (MAC Address) ditambahkan ke data pada lapisan ini, termasuk alamat MAC sumber dan tujuan.

c. **Network**

Network layer menghubungkan host akhir pada jaringan yang berbeda (yaitu, di luar LAN Anda). Lapisan ini menangani alamat logis (Logical address) menggunakan alamat IP.

d. **Transport**

Transport layer menyediakan komunikasi end-to-end antara proses aplikasi. Keandalan, pengurutan, pengendalian aliran, dan pemulihan kesalahan bergantung pada protokol yang digunakan. TCP menyediakan mekanisme tersebut, sedangkan UDP mengirim datagram tanpa menjamin penerimaan atau urutan paket.

e. **Session**

Layer ini mengelola pembentukan, pemeliharaan, sinkronisasi, dan pengakhiran sesi komunikasi antara aplikasi. Dalam implementasi jaringan modern, fungsi session sering digabungkan ke dalam protokol atau aplikasi pada lapisan yang lebih tinggi.

f. **Presentation**

Layer ini memformat data agar dapat dimengerti oleh aplikasi penerima. Layer ini juga dapat mengenkripsi data saat dikirim dan mendekripsi saat diterima, memastikan bahwa hanya penerima yang dimaksud yang dapat membacanya.

g. **Application**

Layer ini paling dekat dengan pengguna dan menyediakan layanan jaringan bagi aplikasi. Contohnya, HTTP digunakan untuk pertukaran sumber daya web, DNS untuk resolusi nama, dan MQTT untuk pertukaran pesan berbasis publish-subscribe.

Model TCP/IP

Model TCP/IP adalah model jaringan yang praktis dan banyak diimplementasikan, menjadi dasar bagi internet modern. Dinamai setelah dua protokol paling terkenal, yaitu Transmission Control Protocol (TCP) dan Internet Protocol (IP). Berbeda dengan model OSI yang memiliki tujuh lapisan, model TCP/IP menyederhanakan arsitektur menjadi empat lapisan, menjadikannya lebih cocok untuk implementasi dan pemecahan masalah di dunia nyata

a. **Network Access & physical atau Link (Ethernet, Wi-Fi, Bluetooth, MAC, VLAN, dsb.)**

Link layer terletak di dasar model TCP/IP dan menjadi antarmuka antara perangkat keras jaringan dengan protokol pada lapisan di atasnya. Lapisan ini mencakup media transmisi, pembingkaian data, pengalamatan lokal, dan akses ke media. Ethernet, Wi-Fi, dan Bluetooth merupakan contoh teknologi yang bekerja pada bagian lapisan akses jaringan, tetapi tingkat keandalannya bergantung pada teknologi dan mekanisme yang digunakan.

b. **Internet (IP, NAT, ARP, dsb.)**

Lapisan Internet bertanggung jawab atas pengalamatan logis dan penerusan paket antarjaringan. Internet Protocol (IP) menyediakan alamat dan format paket, sedangkan router menentukan jalur penerusan berdasarkan tabel routing. IP bersifat best effort, sehingga tidak menjamin bahwa setiap paket sampai, tiba berurutan, atau hanya diterima satu kali. ARP membantu pemetaan alamat IPv4 ke alamat link lokal dan sering ditempatkan pada batas antara lapisan Link dan Internet, bergantung pada model yang digunakan.

c. **Transport (TCP, UDP, RTP, SCTP, DCCP)**

Lapisan Transport menyediakan komunikasi end-to-end antarproses aplikasi. Transmission Control Protocol (TCP) bersifat berorientasi koneksi dan menyediakan pengurutan, retransmisi, serta pengendalian aliran. User Datagram Protocol (UDP) bersifat tanpa koneksi dan memiliki overhead lebih kecil, tetapi tidak menjamin penerimaan, urutan, atau pengiriman tunggal. Pemilihan TCP atau UDP harus disesuaikan dengan kebutuhan aplikasi.

d. **Application (HTTP, DNS, SSH, FTP, TLS, dsb.)**

Di bagian puncak model TCP/IP terdapat Lapisan Aplikasi, berfungsi sebagai antarmuka antara jaringan dan aplikasi pengguna akhir. Lapisan ini mencakup berbagai protokol, masing-masing disesuaikan untuk layanan dan aplikasi tertentu. Contohnya adalah Hypertext Transfer Protocol (HTTP) untuk penjelajahan web, File Transfer Protocol (FTP) untuk pertukaran file, dan Simple Mail Transfer Protocol (SMTP) untuk komunikasi email. Lapisan Aplikasi bertanggung jawab untuk menyediakan layanan jaringan secara langsung kepada pengguna, memungkinkan mereka berinteraksi dan menggunakan berbagai aplikasi yang mendefinisikan pengalaman internet modern. Pada intinya, lapisan ini melengkapi model TCP/IP dengan memfasilitasi komunikasi tanpa hambatan antara aplikasi pengguna dan infrastruktur jaringan yang mendasarinya.

Application Layer

Application Layer pada sistem IoT menyediakan fungsi pemrosesan data, penyimpanan, analitik, aturan otomasi, API, pengelolaan perangkat, serta antarmuka bagi pengguna. Layanan dapat disajikan melalui graphical user interface (GUI), aplikasi seluler, aplikasi web, dashboard, atau integrasi dengan sistem lain.

MODUL 1

KONEKTIVITAS BLUETOOTH

TUJUAN

1) Mengetahui teknologi Bluetooth.
2) Memahami proses komunikasi antarperangkat menggunakan Bluetooth.
3) Mengimplementasikan Bluetooth untuk sistem IoT.

REQUIREMENT

1) Arduino IDE/PlatformIO,
2) ESP32,
3) Sensor (opsional),
4) Aktuator (Opsional).

Konektivitas Bluetooth dapat digunakan untuk komunikasi langsung antardua perangkat. Peer-to-peer (P2P) adalah pola komunikasi ketika perangkat berinteraksi secara langsung tanpa server aplikasi pusat. Namun, istilah peran pada Bluetooth bergantung pada jenis teknologinya: Bluetooth Classic dapat menggunakan peran initiator dan acceptor pada koneksi tertentu, sedangkan Bluetooth Low Energy menggunakan peran Central dan Peripheral pada Generic Access Profile (GAP).

1.1 Bluetooth

Bluetooth adalah sebuah media komunikasi yang dapat digunakan untuk menghubungkan sebuah perangkat komunikasi dengan perangkat komunikasi lainnya tanpa menggunakan kabel. Bluetooth biasa digunakan pada smartphone, laptop, komputer dan lain-lain.

Gambar 1.1: Logo Bluetooth

Teknologi Bluetooth dikembangkan pada dekade 1990-an oleh tim insinyur Ericsson, termasuk Jaap Haartsen. Nama Bluetooth diambil dari julukan Raja Harald "Blåtand" Gormsson, yang dikaitkan dengan penyatuan wilayah Denmark dan Norwegia. Nama tersebut digunakan sebagai simbol penyatuan berbagai perangkat melalui satu teknologi komunikasi nirkabel.



Gambar 1.2: Jacobus Cornelis Haartsen

Bluetooth digunakan untuk pertukaran data jarak dekat dan menghubungkan perangkat seperti headset, speaker, keyboard, sensor, serta komputer. Beberapa profil Bluetooth juga mendukung fungsi jaringan, termasuk tethering.

Bluetooth beroperasi pada pita ISM 2,4 GHz. Bluetooth Classic menggunakan frequency hopping, sedangkan Bluetooth Low Energy menggunakan kanal dan mekanisme akses radio yang

berbeda. Jangkauan, kecepatan, dan konsumsi daya dipengaruhi oleh versi, mode PHY, daya pancar, antena, kondisi lingkungan, serta implementasi perangkat.

1.2 Jenis-Jenis Bluetooth

1.2.1 Bluetooth 1.0

Pada tahun 1999, versi pertama dari Bluetooth ini mengalami banyak masalah dan produsen mengalami kesulitan untuk menciptakan sebuah produk yang bisa saling berhubungan antara satu sama lain dengan benar. Versi 1.0 dan versi perbaikannya 1.0B bisa dibilang mengalami kegagalan.

Lalu pada tahun 2001 Bluetooth 1.1 pun diciptakan dengan perbaikan dari versi pendahulunya. Selain itu, Bluetooth 1.1 juga telah memiliki standar yang dinamakan IEEE 802.15.1-2002.

Pada tahun 2003, Bluetooth 1.2 diperkenalkan dengan peningkatan proses discovery, Adaptive Frequency Hopping (AFH), serta perbaikan kualitas koneksi. Perangkat Bluetooth dirancang untuk mempertahankan interoperabilitas dengan fitur versi sebelumnya, meskipun keberhasilan koneksi tetap bergantung pada profil dan fitur yang didukung oleh kedua perangkat.

1.2.2 Bluetooth 2.0

Pada tahun 2004, Bluetooth 2.0 diperkenalkan dengan Enhanced Data Rate (EDR) sebagai fitur opsional untuk meningkatkan laju data dan efisiensi waktu transmisi. Penggunaan EDR tidak menghilangkan dukungan terhadap Basic Rate, sehingga perangkat dapat tetap berkomunikasi menggunakan fitur yang sama-sama didukung.

Tiga tahun selanjutnya, Bluetooth mengeluarkan versi terbarunya yaitu Bluetooth 2.1. Bluetooth memiliki teknologi baru, yaitu Secure Simple Pairing (SSP) untuk membuat pairing lebih cepat dan aman. Bluetooth juga memiliki teknologi baru yang lainnya yaitu Extended Inquiry Response (EIR), yang membantu pengguna untuk mengetahui informasi perangkat yang akan dihubungkan sebelum terkoneksi dan konsumsi daya lebih hemat.

1.2.3 Bluetooth 3.0

Pada tahun 2009, Bluetooth 3.0 + High Speed (HS) memperkenalkan Alternate MAC/PHY (AMP). Kanal Bluetooth digunakan untuk discovery, koneksi, dan pengelolaan, sedangkan pemindahan data berkecepatan tinggi dapat dialihkan ke radio IEEE 802.11 pada perangkat yang mendukung fitur HS. Laju teoretis yang dipromosikan mencapai sekitar 24 Mbps, tetapi fitur ini bersifat opsional.

1.2.4 Bluetooth 4.0 (Bluetooth Low Energy)

Pada tahun 2010, Bluetooth Core Specification 4.0 memperkenalkan Bluetooth Low Energy (BLE) sebagai sistem radio berdaya rendah di samping Bluetooth Classic. BLE dirancang agar perangkat dapat beroperasi dengan konsumsi energi rendah, termasuk perangkat yang menggunakan baterai kecil. BLE 4.x menggunakan LE 1M PHY dengan laju simbol 1 Msym/s; throughput aplikasi dan jangkauan aktual bergantung pada konfigurasi serta kondisi lingkungan.

Bluetooth 4.1, yang diadopsi pada tahun 2013, meningkatkan koeksistensi dengan jaringan seluler, fleksibilitas peran perangkat, dan pengelolaan koneksi. Perbaikan tersebut tidak berarti koneksi Bluetooth selalu hidup atau mati secara otomatis; perilaku daya tetap ditentukan oleh implementasi perangkat dan aplikasinya.

Bluetooth 4.2, yang diadopsi pada tahun 2014, menambahkan peningkatan pada kapasitas paket data BLE, privasi, keamanan koneksi, dan dukungan Internet Protocol Support Profile.

Peningkatan throughput bergantung pada dukungan fitur Data Length Extension oleh kedua perangkat dan tidak dapat disamakan dengan kecepatan unduh internet.

1.2.5 Bluetooth 5.0

Bluetooth Core Specification 5.0 diadopsi pada tahun 2016 dan memperkenalkan peningkatan penting untuk Bluetooth Low Energy, termasuk LE 2M PHY, LE Coded PHY, dan Extended Advertising. Fitur-fitur tersebut mendukung beragam kebutuhan IoT, namun keamanan dan ketahanan gangguan tetap bergantung pada konfigurasi serta implementasi perangkat.

Tiga fitur utama yang diperkenalkan adalah:

- Hingga 2× laju data melalui LE 2M PHY
- Hingga 4× jangkauan melalui LE Coded PHY pada kondisi tertentu
- Hingga 8× kapasitas data advertising dibandingkan advertising legacy

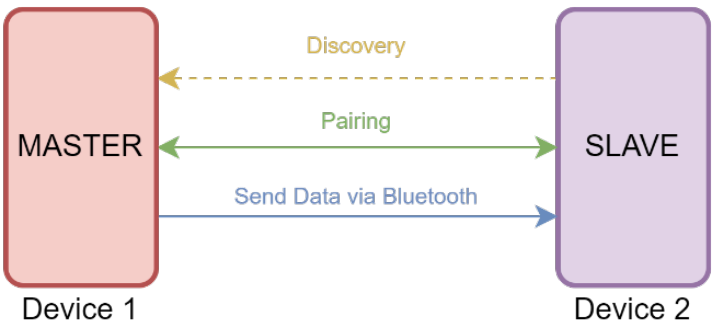
Bluetooth 5.0 tetap mendukung LE 1M PHY dan menambahkan LE 2M PHY untuk laju data lebih tinggi. LE Coded PHY dapat memperluas jangkauan dengan menambahkan redundansi, tetapi menurunkan laju data efektif. Perangkat hanya dapat menggunakan fitur tersebut apabila kedua sisi mendukung dan mengaktifkannya.

1.2.6 Bluetooth 6.0

Versi terbaru dari Bluetooth dirilis pada tahun 2024, berfokus pada ketelitian pada pencarian perangkat, latensi yang lebih rendah, dan peningkatan efisiensi pada pencarian perangkat. Fitur baru dari versi ini meliputi Channel Sounding, dimana perangkat Bluetooth akan menggunakan pencarian berdasarkan fasa serta berdasarkan waktu tiba dan datang dalam pengukuran waktu. Hal ini memungkinkan pencarian perangkat dengan ketelitian jarak sentimeter. Walaupun Bluetooth 6.0 tidak memberikan penambahan performa dalam transfer data, tetapi versi ini memberikan effisensi daya dan keamanan yang lebih baik dibandingkan dengan versi terdahulunya.

1.3 Alur Komunikasi Bluetooth

1.3.1 Bluetooth Serial



Gambar 1.3: Alur Komunikasi Bluetooth Serial

Langkah-langkah Bluetooth Serial bekerja seperti yang ditunjukkan pada gambar di atas, penjelasannya adalah sebagai berikut:

a. **Discovery**

Perangkat melakukan proses discovery untuk menemukan perangkat lain yang berada dalam jangkauan. Pada Bluetooth Classic, proses ini dapat menggunakan prosedur inquiry, sedangkan Bluetooth Low Energy menggunakan scanning terhadap paket advertising.

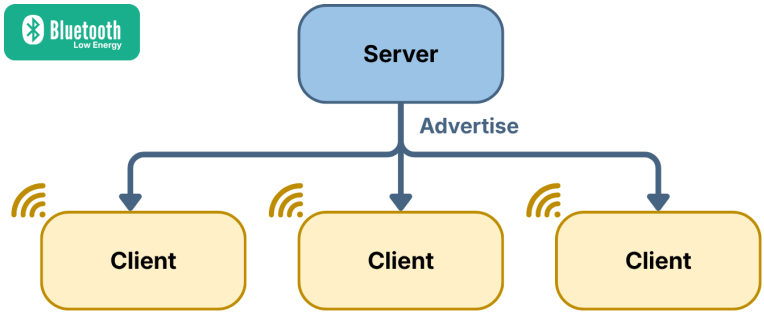
b. **Pairing**

Pairing dapat dilakukan setelah perangkat ditemukan apabila aplikasi memerlukan pembentukan kunci dan hubungan tepercaya. Tidak semua koneksi Bluetooth mewajibkan pairing; kebutuhan keamanan ditentukan oleh profil, layanan, dan konfigurasi perangkat.

c. **Connection**

Perangkat kemudian dapat membangun koneksi untuk bertukar data. Peran koneksi tidak menunjukkan arah pengiriman data: kedua perangkat dapat mengirim dan menerima. Pada Bluetooth Low Energy digunakan istilah Central dan Peripheral, sedangkan istilah peran pada Bluetooth Classic bergantung pada prosedur dan profil yang digunakan.

1.3.2 Bluetooth Low Energy



Gambar 1.4: Alur Komunikasi Bluetooth Low Energy

Berbeda dengan Bluetooth Serial, Bluetooth Low Energy menggunakan konsep **Server/ Peripheral** dan **Client/Central**:

a. **Server**

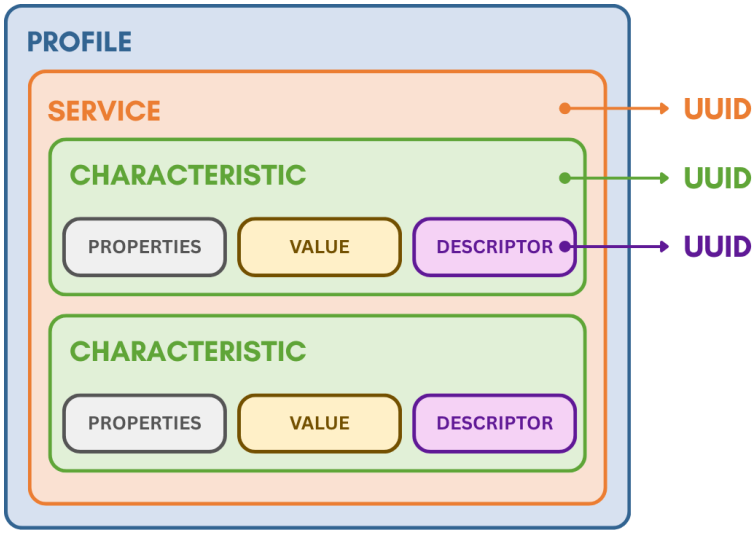
Server akan melakukan advertise atau mengiklankan dirinya supaya dapat diketahui oleh perangkat yang terkonfigurasi sebagai Client. Server dapat berupa perangkat yang digunakan untuk mengirimkan data dari hasil akuisisi data.

b. **Client**

Client akan melakukan scanning dengan perangkat yang terkonfigurasi sebagai server. Ketika Client menemukan Server yang diinginkan, maka koneksi akan dilakukan dan akan terus mendengarkan data yang terkirim dari Server. Komunikasi tersebut dinamakan point-to-point communication.

Dalam komunikasi BLE, ada dua mode topologi komunikasi yang dapat digunakan:

- **Broadcast Mode**: Server akan melakukan pengiriman data ke Client yang terkoneksi. Koneksi ini disebut one-to-many connection.
- **Mesh Network**: Semua perangkat terhubung baik Client maupun Server satu sama lain. Koneksi ini disebut many-to-many connection.



Gambar 1.5: Struktur GATT

Generic Attributes (GATT) digunakan pada BLE sebagai hirarki struktur data yang digunakan dalam komunikasi antar perangkat BLE. Tingkat paling atas dari struktur GATT adalah **Profile**, dimana terdiri dari berbagai satu atau lebih Service. Setiap **Service** terdiri setidaknya satu **Characteristic** atau bisa berupa referensi Service lainnya. Service berbentuk koleksi informasi seperti pembacaan sensor.

Characteristic akan selalu dimiliki oleh satu **Service** dan merupakan letak dimana data akan disimpan di dalam hirarki (value). Characteristic memiliki dua attribute, yakni:

- **Characteristic Declaration**: Menyediakan metadata dari data yang dikirim
- **Characteristic Value**: Nilai dari data itu sendiri.

Setelah itu, Characteristic Value dapat diikuti oleh **Descriptor**, dimana akan memberikan informasi lebih lanjut dari metadata yang tersimpan di dalam Characteristic Declaration.

Properties menjelaskan interaksi Characteristic Value yang dapat dilakukan. Pada dasarnya, Properties menyediakan operasi dan prosedur yang bisa digunakan oleh Characteristic sebagai berikut:

- Broadcast
- Read
- Write without Response
- Write
- Notify
- Indicate
- Authenticated Signed Wires
- Extended Properties

Di dalam setiap Service, Characteristic, dan Descriptor, terdapat **Universally Unique Identifier (UUID)** yang berisi angka unik 128-bit (16 bytes). UUID dapat dibuat melalui UUID generator. UUID digunakan untuk dapat mengidentifikasi informasi unik yang diiginkan, seperti Service yang disediakan oleh perangkat BLE.

1.4 Protokol Bluetooth

Bluetooth Special Interest Group (SIG) telah mengembangkan spesifikasi Bluetooth yang berisi tentang protokol yang akan digunakan dalam teknologi Bluetooth ini. Protokol dasar

Bluetooth adalah Bluetooth Radio, Baseband dan Link Manager Protocol (LMP) yang disebut protokol inti. Sedangkan protokol yang ada di atasnya adalah protokol-protokol terapan yang dapat diadaptasikan pada arsitektur protokol Bluetooth dan telah dikembangkan oleh organisasi lain seperti ETSI. Radio, baseband dan LMP ekivalen dengan lapis fisik dan data link pada lapis protokol OSI.

Stack protokol Bluetooth dapat dibagi ke dalam empat layer sesuai dengan tujuannya. Berikut protokol-protokol dalam layer-layer di dalam stack protokol Bluetooth yang tertera pada tabel di bawah ini:

Tabel 1.1: Layer Protokol dan stack protokol Bluetooth

Layer Protokol	**Protokol di Stack**
Bluetooth Core Protocols	Baseband, LMP, L2CAP, SDP
Cable Replacement Protocols	RFCOMM
Telephony Control Protocols	TCS Binary, AT-commands
Adopted Protocols	PPP, UDP/TCP/IP, OBEX, WAP, vCard, vCal, IrMC, WAE

Selain itu juga terdapat Host Controller Interface (HCI) yang melakukan perintah-perintah yang menghubungkan control baseband, pengaturan hubungan yang dibangun, dan akses ke hardware dan pengaturan register.

a. **Baseband**

Lapis yang memungkinkan hubungan RF terjadi antara beberapa unit Bluetooth membentuk piconet. Sistem RF dari Bluetooth ini menggunakan frekuensi-hopping-spread spectrum yang mengirimkan data dalam bentuk paket pada time slot dan frekuensi yang telah ditentukan, lapis ini melakukan prosedur pemeriksaan dan paging untuk sinkronisasi transmisi frekuensi hopping dan clock dari perangkat Bluetooth yang berbeda.

b. **Link Manager Protocol (LMP)**

Link Manager Protocol mengatur pembentukan dan pengelolaan tautan pada Bluetooth Classic. Fungsinya mencakup autentikasi, pengaturan enkripsi, negosiasi parameter tautan, dan pengendalian fitur baseband.

c. **Logical Link Control and Adaptation Protocol (L2CAP)**

Paket L2CAP membawa muatan yang penting yang dibawa ke layer protokol yang lebih tinggi.

d. **Service Discovery Protocol (SDP)**

Protokol ini digunakan untuk menemukan layanan yang disediakan oleh perangkat Bluetooth Classic beserta atribut yang diperlukan untuk mengakses layanan tersebut.

e. **Cable Replacement Protocol (RFCOMM)**

RFCOMM adalah emulasi jalur serial.

f. **Telephony Control Protocol**

Telephony Control Specification Binary (TCS BIN) dan perintah AT digunakan pada profil telepon tertentu untuk mengendalikan panggilan serta fungsi modem.

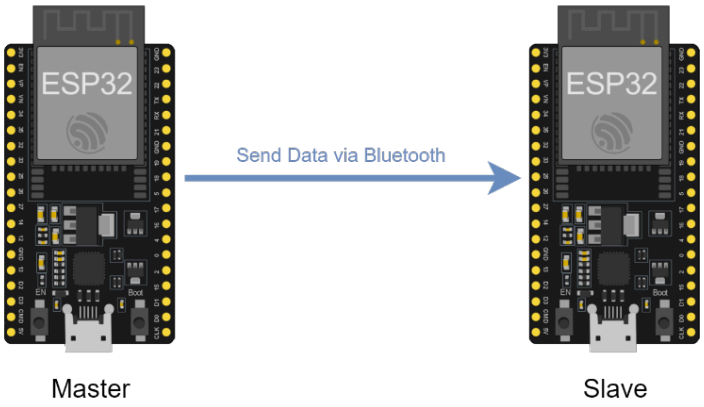
g. **Adopted Protocols**

Bluetooth juga mendukung protokol PPP, TCP/UDP/IP, OBEX dan WAP untuk memaksimalkan interoperabilitasnya.

h. **Radio Frequency (RF)**

Radio Frequency merupakan lapisan terendah dari spesifikasi Bluetooth dan menyediakan komunikasi nirkabel pada pita Industrial, Scientific, and Medical (ISM) 2,4 GHz. Bluetooth Classic menggunakan 79 kanal RF dengan lebar 1 MHz dan mekanisme frequency hopping, sedangkan Bluetooth Low Energy menggunakan 40 kanal RF dengan lebar 2 MHz.

1.5 Implementasi peer-to-peer Bluetooth



Gambar 1.6: Visual P2P

Dalam implementasi peer-to-peer Bluetooth ini, master akan mengirimkan data ke slave menggunakan mikrokontroler ESP32. Jika ingin menggunakan mikrokontroller yang lain, pastikan perangkat mensupport jenis Bluetooth yang ingin dipakai. Bila perangkat hanya mensupport Bluetooth versi 4.0 ke atas tanpa adanya dual-mode, maka perangkat tersebut hanya dapat menggunakan BLE dan begitu sebaliknya. Bila perangkat memiliki dual-mode support (Bluetooth Classic dan BLE), maka perangkat tersebut dapat menggunakan implementasi Bluetooth Classic dan BLE.

1.5.1 Menggunakan Bluetooth Serial

Konfigurasi Bluetooth Serial tergolong sederhana dibandingkan dengan BLE karena menggunakan konsep **Master** dan **Slave** seperti protokol serial komunikasi lainnya. Berikut adalah contoh konfigurasi master dan slave pada ESP32.

Informasi
Library Bluetooth Serial yang akan dipakai adalah library builtin dari ESP32:
https://github.com/espressif/arduino-esp32/tree/master/libraries/BluetoothSerial

- **Master**

Master merupakan bagian yang akan mencari koneksi kepada Slave dan akan mengirimkan data kepada mereka.

Inisialisasi Source code untuk Master/Host

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;
```

```
4
5  String masterName = "ESP32-Master"; // Nama Master ESP
6  String slaveName = "ESP32-Slave";
7  bool connected = false;
8  int value = 0;
```

Dalam menginisialisasi P2P ESP32, kita menggunakan library `BluetoothSerial.h` agar bisa mengakses kode-kode yang mendukung fungsi Bluetooth. Setelah itu, siapkan variabel-variabel seperti variabel untuk mengakses fungsi Bluetooth seperti variabel `SerialBT` yang dapat mengakses BluetoothSerial, variabel `String masterName` yang digunakan untuk mencari nama perangkat yang dituju yaitu `ESP32-Slave`, `bool connected` untuk konektivitas dan `int value` untuk digunakan sebagai penyimpan nilai yang akan dikirimkan ke Slave.

Setup pada Source code Master

```
1  void setup() {
2    Serial.begin(115200);
3
4    SerialBT.begin(masterName, true); // Memulai Bluetooth
5    Serial.println("The device started in master mode, make sure remote BT device is on!");
6
7    connected = SerialBT.connect(slaveName); // Periksa koneksi Bluetooth
8    if(connected) {
9      Serial.println("Connected Successfully!");
10   } else {
11     while(!SerialBT.connected(10 * 1000)) { // Menunggu 10 detik
12       Serial.println("Failed to connect!");
13       Serial.println("Make sure remote device is available and in range, then restart");
14     }
15   }
16   SerialBT.connect();
17 }
```

Pada `setup()`, kita menggunakan `SerialBT.begin("ESP32Master", true)` untuk menentukan nama yang akan terlihat oleh perangkat lain. Untuk `connected = SerialBT.connect(Name)` digunakan sebagai koneksi kepada Bluetooth yang dituju dan indikator untuk terkoneksi atau tidak. Dan dilanjutkan dengan menghubungkan dengan Bluetooth pada `SerialBT.connect();`

Loop pada Source Code Master

```
1  void loop() {
2    if(value <= 10){
3      SerialBT.println(value); // Mengirimkan data melalui Bluetooth
4      Serial.println(value);
5      value++;
6
7      delay(1000);
8    }
9    else {
10     value = value - 10;
11   }
12 }
```

Pada tahap perulangan, kita akan mencoba mengirimkan kepada `Slave` nilai dari variabel `value` hingga bernilai 10. Kita menggunakan `SerialBT.println(value)` yang di mana digunakan untuk mengirimkan nilai kepada `Slave` berdasarkan nilai dari variabel `value`.

- **Slave**

  Slave merupakan bagian yang akan menerima komunikasi dari Master.

Inisialisasi Source code untuk Slave

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // Nama Slave ESP
int value = 0;
String slaveValueStr;
int slaveValueInt;
```

Pada proses inisialisasi pada Slave, kita kembali menggunakan library `BluetoothSerial.h`.. Kemudian, inisialisasikan kembali `BluetoothSerial` pada variabel `SerialBT` dan siapkan variabel-variabel yang akan digunakan seperti `int value`, `String slaveValueStr` yang akan digunakan sebagai data yang diterima berupa string, dan `int slaveValueInt` yang akan digunakan untuk menyimpan nilai int yang telah diubah dari string ke int. Terakhir, `String masterName = "ESP32-BT-Slave"` merupakan inisial untuk nama perangkat kita.

Setup pada Source code Slave

```
void setup() {
  Serial.begin(115200);

  SerialBT.begin(slaveName); // Memulai Bluetooth sebagai perangkat penerima
  Serial.println("The device started, now you can pair it with bluetooth!");
}
```

Pada setup, kita akan mencoba menyambungkan koneksi dengan `SerialBT.begin(slaveName)` dan menentukan nama yang akan terlihat oleh perangkat lain berdasarkan nilai dari variabel `slaveName`.

Loop pada Source code Slave

```
void loop() {
  if(SerialBT.available()){
    slaveValueStr = SerialBT.readStringUntil('\n'); // Membaca data dari Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data dari Master : ");
    Serial.println(slaveValueInt);
  }
}
```

Pada perulangan, kita mencoba untuk memeriksa apakah Bluetooth sudah terhubung dengan perangkat master atau tidak dengan `if(SerialBT.available())`. Kemudian, variabel `slaveValueStr` akan membaca data string hingga akhir kalimat sebelum baris baru. Dari Master, kita menggunakan `SerialBT.readStringUntil('\n')`, lalu kita akan mengubah nilai `String` yang dibaca dari master menjadi `int` yang akan disimpan ke variabel `slaveValueInt` menggunakan `slaveValueStr.toInt()` sehingga akan menampilkan hasil Serial Monitor berikut.

```
Data Dari Master : 1
Data Dari Master : 2
Data Dari Master : 3
Data Dari Master : 4
Data Dari Master : 5
Data Dari Master : 6
Data Dari Master : 7
Data Dari Master : 8
Data Dari Master : 9
Data Dari Master : 10
Data Dari Master : 1
```

Gambar 1.7: Output pada Serial Monitor

Gambar di atas merupakan tampilan Serial Monitor pada Slave jika menerima data yang dikirimkan dari Master berhasil.

1.5.2 Menggunakan Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) menggunakan model **Central** dan **Peripheral**. Perangkat **Central/Client** melakukan pemindaian dan menghubungkan diri ke perangkat BLE, sedangkan **Peripheral/Server** melakukan **advertising** serta menyediakan data melalui struktur GATT (**Generic Attribute Profile**).

Pada contoh ini, **Peripheral/Server** menghasilkan nilai acak dan mengirimkannya melalui notifikasi. **Central/Client** mencari perangkat dengan **Service UUID** yang sesuai, melakukan koneksi, lalu menerima data melalui fungsi callback.

Informasi
Untuk mendapatkan nilai UUID random, website di bawah dapat digunakan:
https://uuidgenerator.net

- **Central/Client**

Central bertugas memindai perangkat BLE, menemukan Peripheral berdasarkan **Service UUID**, melakukan koneksi, dan menerima notifikasi dari characteristic.

Library, Variabel, dan Deklarasi BLE Central/Client

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress = nullptr;
static BLERemoteCharacteristic *pRemoteCharacteristic = nullptr;
static BLEClient *pClient = nullptr;

static bool doConnect = false;
static bool connected = false;
```

Library tersebut menyediakan fungsi scanning, koneksi client, dan akses ke characteristic pada BLE. Variabel pointer menyimpan alamat perangkat, objek client, dan characteristic remote. Sementara itu, `doConnect` dan `connected` mengatur proses serta status koneksi.

Fungsi Notifikasi BLE Central/Client

```
static void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify) {

  String receivedData = "";
  for (size_t i = 0; i < length; i++) {
    receivedData += (char)pData[i];
  }

  Serial.print("Received from Slave: ");
  Serial.println(receivedData);
}
```

Fungsi `notifyCallback()` dijalankan secara otomatis ketika Central menerima notifikasi dari characteristic milik Peripheral. Parameter `pData` berisi data yang diterima dalam bentuk array byte, sedangkan `length` menunjukkan jumlah byte yang diterima. Setiap byte diubah menjadi karakter menggunakan `static_cast<char>()`, kemudian digabungkan ke dalam variabel `receivedData`.

Setelah seluruh data selesai diproses, nilainya ditampilkan melalui Serial Monitor.

Callback Koneksi BLE Central/Client

```
class MyClientCallbacks : public BLEClientCallbacks {
  void onConnect(BLEClient *pClient) override {
    connected = true;
    Serial.println("Connected to Peripheral!");
  }

  void onDisconnect(BLEClient *pClient) override {
    connected = false;
    Serial.println("Disconnected from Peripheral!");
  }
};
```

Kelas `MyClientCallbacks` digunakan untuk mendeteksi perubahan status koneksi antara Central dan Peripheral. Fungsi `onConnect()` dijalankan ketika Central berhasil terhubung ke Peripheral. Pada kondisi ini, variabel `connected` diubah menjadi `true`. Fungsi `onDisconnect()` dijalankan ketika koneksi terputus. Variabel `connected` diubah menjadi `false` agar Central dapat menjalankan proses scanning ulang.

Callback Scanning BLE Central/Client

```
class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
 void onResult(BLEAdvertisedDevice advertisedDevice) override {
    Serial.print("Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    // Cek apakah device memiliki service UUID yang sesuai
    if (
      advertisedDevice.haveServiceUUID() &&
      advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ) {
      Serial.println("Target Peripheral found!");
      BLEDevice::getScan()->stop();
```

```
13
14        if (pServerAddress != nullptr) {
15          delete pServerAddress;
16        }
17
18        pServerAddress = new BLEAddress(advertisedDevice.getAddress());
19
20        doConnect = true;
21      }
22    }
23  };
```

Kelas `MyAdvertisedDeviceCallbacks` digunakan untuk memproses setiap perangkat BLE yang ditemukan selama scanning. Fungsi `onResult()` dijalankan setiap kali scanner menemukan perangkat BLE. Informasi perangkat tersebut ditampilkan melalui `advertisedDevice.toString()`. Fungsi `haveServiceUUID()` memeriksa apakah perangkat mengiklankan Service UUID. Setelah itu, `isAdvertisingService()` membandingkan UUID perangkat dengan `SERVICE_UUID` yang dicari.

Jika UUID sesuai, scanning dihentikan menggunakan `BLEDevice::getScan()->stop()`. Alamat Peripheral disimpan dalam `pServerAddress`, kemudian `doConnect` diubah menjadi `true` agar proses koneksi dijalankan pada bagian `loop()`.

Inisialisasi dan Koneksi BLE Central/Client

```
1   bool connectToServer(BLEAddress address) {
2     Serial.print("Connecting to Peripheral: ");
3     Serial.println(address.toString().c_str());
4
5     pClient = BLEDevice::createClient();
6     pClient->setClientCallbacks(new MyClientCallbacks());
7
8     if (!pClient->connect(address)) {
9       Serial.println("Connection failed!");
10      return false;
11    }
12
13    BLERemoteService *pRemoteService = pClient->getService(BLEUUID(SERVICE_UUID));
14   if (pRemoteService == nullptr) {
15      Serial.println("Service not found!");
16      pClient->disconnect();
17      return false;
18    }
19
20    pRemoteCharacteristic = pRemoteService->getCharacteristic(BLEUUID(CHARACTERISTIC_UUID));
21   if (pRemoteCharacteristic == nullptr) {
22      Serial.println("Characteristic not found!");
23      pClient->disconnect();
24      return false;
25    }
26
27    if (!pRemoteCharacteristic->canNotify()) {
28      Serial.println("Characteristic does not support notifications!");
29      pClient->disconnect();
30      return false;
31    }
32
33    pRemoteCharacteristic->registerForNotify(notifyCallback);
34    connected = true;
35
36    Serial.println("Notification enabled!");
37    return true;
38  }
```

`connectToServer()` menghubungkan Central, mencari service dan characteristic, lalu mendaftarkan callback notifikasi. Fungsi ini akan digunakan di fungsi `loop()` nanti.

Setup pada Source Central/Client

```
void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Central/Client ===");

  // Inisialisasi BLE Master
  BLEDevice::init("ESP32_Central");

  // Mulai Scanning
  BLEScan *pBLEScan = BLEDevice::getScan();
  pBLEScan->setAdvertisedDeviceCallbacks(
    new MyAdvertisedDeviceCallbacks()
  );
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);

  Serial.println("Scanning for Peripheral...");
  pBLEScan->start(5, false); // Scan selama 5 detik
}
```

Pada `setup()`, kita menginisialisasi Serial dan BLE dan melakukan registrasi callback yang telah kita buat ke dalam pointer `pBLEScan` melalui `setAdvertisedDeviceCallbacks()`. Selain itu, interval pengiriman dan proses pencarian peripheral juga dilaksanakan.

Loop BLE Central/Client

```
void loop() {
  // Jika menemukan device yang cocok, coba koneksi
  if (doConnect && pServerAddress != nullptr) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Ready to receive data!");
    } else {
      Serial.println("Connection failed. Scanning again...");
    }

    doConnect = false;
  }

  // Jika terputus, coba scan lagi
  if (!connected && !doConnect) {
    BLEDevice::getScan()->start(5, false);
  }

  delay(1000);
}
```

Pada `loop()`, Central mencoba terhubung setelah target ditemukan. Jika belum terhubung, proses scanning akan dijalankan kembali.

- **Peripheral/Server**

Peripheral melakukan **advertising**, menyediakan service dan characteristic, lalu mengirim nilai acak kepada Central melalui notifikasi.

Library, Variabel, dan Deklarasi BLE Peripheral/Server

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
```

```
#include <BLE2902.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

BLEServer *pServer = nullptr;
BLECharacteristic *pCharacteristic = nullptr;

bool deviceConnected = false;
bool oldDeviceConnected = false;

unsigned long previousSendTime = 0;
const unsigned long sendInterval = 1000;
```

Library BLE Server digunakan untuk membuat server, service, characteristic, dan descriptor notifikasi. Variabel koneksi menyimpan status Central, sedangkan variabel waktu mengatur interval pengiriman data.

Callback BLE Peripheral/Server

```
class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) override {
    deviceConnected = true;
    Serial.println("Central connected!");
  }

  void onDisconnect(BLEServer *pServer) override {
    deviceConnected = false;
    Serial.println("Central disconnected!");
  }
};
```

Callback server dijalankan saat Central terhubung atau terputus. Nilai `deviceConnected` digunakan oleh `loop()` untuk menentukan apakah notifikasi dapat dikirim.

Inisialisasi BLE Peripheral/Server

```
void BLE_init() {
  BLEDevice::init("ESP32_Peripheral");

  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService =
    pServer->createService(BLEUUID(SERVICE_UUID));

  // Membuat characteristic (data) di dalam service
  // PROPERTY_READ     -> Client bisa baca data
  // PROPERTY_WRITE    -> Client bisa kirim data ke ESP32
  // PROPERTY_NOTIFY   -> ESP32 kirim data otomatis (tanpa ACK)
  // PROPERTY_INDICATE -> ESP32 kirim data otomatis (dengan ACK)
  // PROPERTY_WRITE_NR -> Client kirim data tanpa konfirmasi
  pCharacteristic = pService->createCharacteristic(
    BLEUUID(CHARACTERISTIC_UUID),
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setValue("0");

  pService->start();

  BLEAdvertising *pAdvertising =
```

```
    BLEDevice::getAdvertising();

  pAdvertising->addServiceUUID(BLEUUID(SERVICE_UUID));
  pAdvertising->setScanResponse(true);

  BLEDevice::startAdvertising();

  Serial.println("Peripheral ready. Waiting for Central...");
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  BLE_init();
}
```

`BLE_init()` membuat server, service, dan characteristic, menambahkan descriptor notifikasi, lalu memulai advertising. Fungsi `setup()` akan menyiapkan Serial, generator nilai acak untuk percobaan pengiriman data, dan konfigurasi BLE.

Loop BLE Peripheral/Server

```
void loop() {
  if (
    deviceConnected &&
    millis() - previousSendTime >= sendInterval
  ) {
    previousSendTime = millis();

    String value = String(random(0, 101));

    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();

    Serial.print("Sent: ");
    Serial.println(value);
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();

    Serial.println("Advertising restarted...");
    oldDeviceConnected = false;
  }

  if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = true;
  }
}
```

Pada `loop()`, Peripheral mengirim nilai acak setiap satu detik ketika Central terhubung. Jika koneksi terputus, advertising diaktifkan kembali agar perangkat dapat ditemukan dan dihubungkan ulang.

Dengan alur tersebut, Peripheral mengirimkan data dan Central menerimanya melalui notifikasi. Komunikasi tidak menggunakan aliran serial seperti Bluetooth Classic, tetapi menggunakan service, characteristic, dan callback pada GATT.

MODUL 2

KOMUNIKASI WEBSOCKET DENGAN SERVER LOKAL

TUJUAN

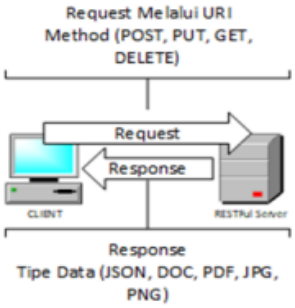
1) Mengetahui protokol HTTP (Hypertext Transfer Protocol).
2) Memahami komunikasi protokol HTTP.
3) Memahami penggunaan WebSocket dalam konteks sistem IoT.

REQUIREMENT

1) Arduino IDE/PlatformIO.
2) ESP32/ESP8266.
3) Server Lokal (Raspberry Pi 3B).
4) Sensor (opsional).
5) Aktuator (opsional).

2.1 Hypertext Transfer Protocol (HTTP)

HTTP (Hypertext Transfer Protocol) adalah protokol lapisan aplikasi untuk pertukaran representasi sumber daya antara client dan server. Client mengirimkan request yang berisi metode, target, header, dan bila diperlukan body. Server kemudian mengirimkan response yang berisi status, header, dan representasi data. HTTP dapat digunakan untuk dokumen, data terstruktur, gambar, video, maupun layanan API, baik melalui jaringan lokal maupun internet. Browser web merupakan salah satu client HTTP, tetapi HTTP juga digunakan oleh aplikasi seluler, perangkat IoT, dan layanan antarsistem.



Gambar 2.1: Visualisasi HTTP

2.1.1 CRUD

CRUD adalah singkatan dari Create, Read, Update, dan Delete. Istilah ini menjelaskan empat operasi dasar terhadap data dan dapat diterapkan pada berbagai jenis penyimpanan, tidak terbatas pada basis data relasional. Pada sistem berbasis HTTP, operasi CRUD sering dipetakan ke metode HTTP, tetapi semantik metode HTTP tetap mengikuti spesifikasi protokol dan tidak selalu sama persis dengan operasi basis data.

Fungsi-fungsi dalam CRUD adalah sebagai berikut.

- **CREATE :** Untuk membuat entri atau record baru dalam sebuah tabel di dalam database. Di dalam aplikasi berbasis SQL, fungsi create sering kali disebut dengan insert
- **READ :** Untuk membaca entri data yang sudah terdaftar dalam sebuah database. Namun, fungsi ini tidak akan memberikan akses untuk mengubah data yang telah tersimpan di dalam database.
- **UPDATE :** Untuk memperbarui entri data ketika informasi di dalamnya memerlukan perubahan. Dengan fungsi ini, Anda dapat mengubah detail dari sebuah entri di database.
- **DELETE :** Untuk menghapus entri data yang sudah tidak diperlukan dalam sebuah database. Ketika menggunakan fungsi ini, Anda akan mengakses detail terkait sebuah entri dan kemudian memberikan perintah kepada sistem untuk menghilangkannya dari database.

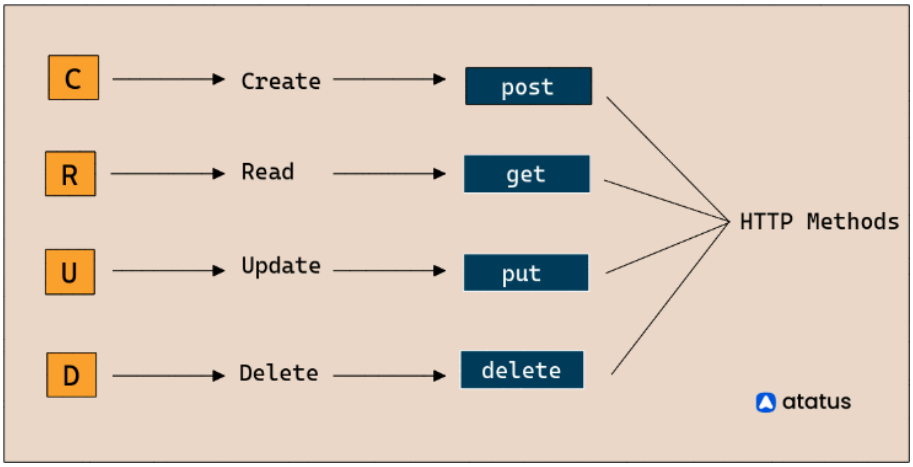
Terdapat beberapa variasi istilah yang dikembangkan dari konsep CRUD. Istilah-istilah berikut bukan bagian dari standar HTTP, melainkan pola penamaan operasi aplikasi:

- BREAD (Browse, Read, Edit, Add, Delete)
- CRAP (Create, Replicate, Append, dan Process)
- CRUDL (Create, Read, Update, Delete, List)
- DAVE (Delete, Add View, Edit)

2.1.2 Metode Komunikasi Protokol HTTP

Metode HTTP memiliki semantik tersendiri dan dapat digunakan untuk mendukung operasi CRUD pada API:

- **GET** meminta representasi sumber daya dan tidak seharusnya mengubah keadaan server.
- **POST** mengirimkan data untuk diproses sesuai fungsi sumber daya; metode ini sering digunakan untuk membuat entri baru.
- **PUT** membuat atau mengganti representasi sumber daya pada target yang ditentukan.
- **PATCH** menerapkan perubahan sebagian terhadap sumber daya.
- **DELETE** meminta penghapusan hubungan sumber daya pada target yang ditentukan.



Gambar 2.2: Metode Komunikasi HTTP

Selain dari metode yang telah disebutkan sebelumnya, ada beberapa metode komunikasi yang dapat digunakan pada protokol HTTP seperti yang ditunjukkan pada tabel berikut.

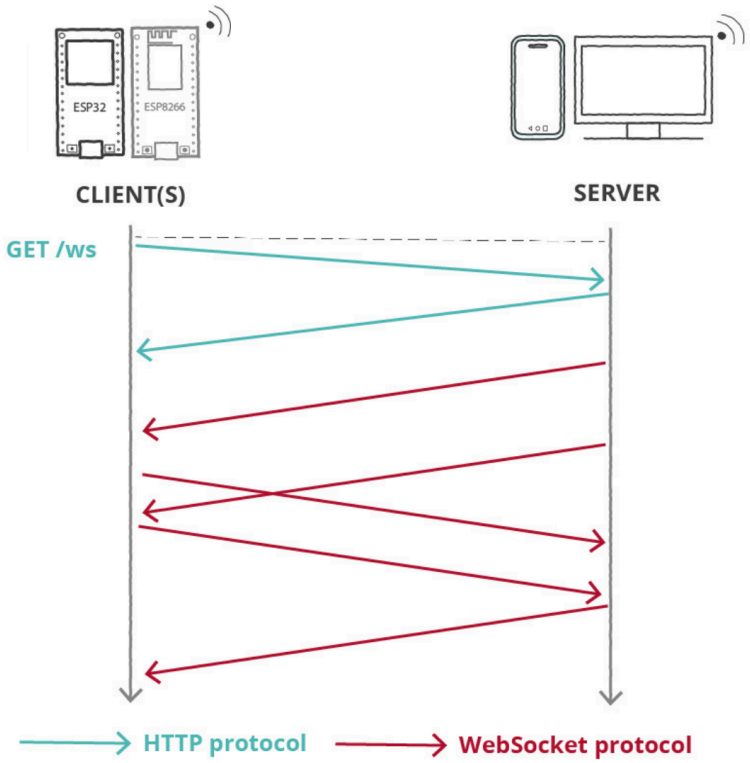
Tabel 2.1: Jenis dan Fungsi Tiap Metode HTTP

Metode HTTP	Definisi	Penjelasan
GET	Mengambil Sumber daya	Digunakan untuk mengambil data tertentu dari server
POST	Memproses data	Digunakan untuk mengirim data agar diproses sesuai fungsi sumber daya
PUT	Membuat atau mengganti sumber daya	Digunakan untuk membuat atau mengganti representasi sumber daya pada target
DELETE	Menghapus sumber daya	Digunakan untuk menghapus data tertentu dari server
PATCH	Mengubah bagian sumber daya	Digunakan untuk memperbarui atau mengubah bagian dari data yang sudah ada pada server
HEAD	Mendapatkan informasi header	Digunakan untuk meminta informasi header dari data yang diminta melalui metode GET
OPTIONS	Mendapatkan informasi dari server	Digunakan untuk meminta informasi tentang metode-metode yang didukung oleh server untuk data tertentu
CONNECT	Membentuk tunnel	Digunakan untuk meminta proxy membentuk tunnel ke server tujuan

TRACE	Melakukan diagnostik	Digunakan untuk meminta penerima akhir mengirimkan kembali request yang diterima

2.2 WebSocket

WebSocket adalah protokol yang menyediakan komunikasi dua arah melalui satu koneksi TCP. Koneksi diawali dengan HTTP Upgrade handshake, kemudian client dan server dapat saling mengirim frame tanpa membuat request HTTP baru untuk setiap pesan. WebSocket sesuai untuk aplikasi yang memerlukan pertukaran data interaktif atau berfrekuensi tinggi, tetapi latensi aktual tetap dipengaruhi oleh jaringan, implementasi, dan beban server.



Gambar 2.3: Alur Komunikasi WebSocket

Berikut adalah alur komunikasi WebSocket:

a. Client mengirimkan request HTTP khusus untuk meminta peningkatan koneksi menjadi WebSocket. Berikut contoh header request:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://example.com
```

Ketika server menerima permintaan tersebut, maka server akan merespons dengan:

```
HTTP/1.1 101 Switching Protocols
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

b. Setelah handshake berhasil, client dan server dapat saling mengirim frame. Frame WebSocket tetap memiliki header protokol, tetapi tidak membawa seluruh header HTTP pada setiap pesan. Struktur frame memuat unsur berikut:
   - **FIN bit** : Indikator apakah frame tersebut merupakan akhir dari pesan.
   - **Opcode** : Tipe dari frame (text, binary, close, ping, pong).
   - **Mask bit** : Menunjukkan apakah payload diberi masking. Frame dari client ke server wajib dimasking, sedangkan frame dari server ke client tidak dimasking. Masking bukan enkripsi dan tidak memberikan kerahasiaan.
   - **Payload length** : Ukuran dari data pesan yang dikirim
   - **Payload data** : Konten data yang dikirim

Berikut adalah perbedaan antara komunikasi WebSocket dengan komunikasi HTTP tradisional:

Tabel 2.2: Perbedaan WebSocket dengan HTTP Tradisional

Perbedaan	HTTP	WebSocket
Pola Komunikasi	Menggunakan pola request-response; umumnya client memulai pertukaran	Mempertahankan koneksi setelah handshake sehingga client dan server dapat saling mengirim pesan
Overhead Koneksi	Setiap request dan response membawa header HTTP; ukurannya bergantung pada konfigurasi	Setelah handshake, overhead frame bervariasi menurut panjang payload, masking, dan ekstensi
Latensi	Memerlukan komunikasi bolak-balik di setiap interaksinya	Mengurangi kebutuhan handshake atau request baru untuk setiap pesan, tetapi tidak menghilangkan latensi jaringan
Skalabilitas	Oleh karena sifatnya stateless, maka skalabilitas secara horizontal menjadi lebih mudah	Koneksi WebSocket sifatnya stateful, sehingga lebih kompleks dari segi skalabilitas

2.3 Implementasi Protokol WebSocket

2.3.1 Server Python Raspberry Pi

Sebelum melakukan komunikasi, kita perlu membuat server WebSocket agar ESP32 dapat terhubung dan bertukar pesan. Buatlah folder baru untuk menyimpan kode server yang akan dipakai, misal dengan nama `server-websocket`. Instal paket Python yang diperlukan terlebih dahulu jika belum terinstal:

Instalasi paket python dan virtual environment

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Berikutnya, buatlah virtual environment di dalam folder yang sudah dibuat sebagai berikut:

Membuat virtual environment python

```
python3 -m venv .venv

# Mengaktifkan virtual environment
source .venv/bin/activate
```

Setelah masuk ke dalam virtual environment, instal paket `websockets` pada Python sebagai berikut di terminal:

Instalasi paket WebSocket Python

```
pip install websockets
```

Buatlah kode sumber python di dalam folder tersebut, misal dengan nama `main.py`. Isi dengan konten berikut:

Import library dan konfigurasi variabel

```
import asyncio  # Mendukung eksekusi coroutine secara asinkron

# Pustaka WebSocket
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0"  # Mendengarkan pada semua antarmuka jaringan
PORT: int = 8765
```

Fungsi callback untuk menerima beberapa client

```
async def handle_client(websocket: ServerConnection) -> None:
    # Di sini, kita akan membedakan tipe pesan yang dikirim pada websocket
    # Untuk path yang akan dipakai saat ini adalah "/esp32"
    path: str = websocket.request.path

    # Kita dapat atur path yang kita ingin proses
    if path == "/esp32":
        print("ESP32 device connected")
        try:
            async for message in websocket:
                # Message from ESP32
                print( f"ESP32 message: {message}" )

                # Reply
                response: str = ( f"ACK from Raspberry Pi: {message}" )
                await websocket.send(response)

        except ConnectionClosed:
            print("ESP32 disconnected")
    # Anda dapat tambah path-path yang lain sesuai kebutuhan Anda
    # Case default ketika tidak sesuai
    else:
        print(f"Unknown path: {path}")
```

Pada kode di atas, kata kunci `async` mendefinisikan coroutine yang dapat dijalankan secara asinkron atau dalam artian lain tidak berurutan. Objek `websocket` mewakili satu koneksi client. Pesan diterima melalui perulangan `async for`, sedangkan metode `websocket.send()` digunakan untuk mengirim balasan pada koneksi tersebut.

Fungsi main pada Python

```
async def main() -> None:
    # Isi dengan konfigurasi yang telah kita buat
    async with serve(handle_client, HOST, PORT):
        print(f"WebSocket server running " f"on {HOST}:{PORT}")
        # Menunggu pesan...
        await asyncio.Future()

if __name__ == "__main__":
    asyncio.run(main())
```

Kata kunci `await` menangguhkan coroutine saat menunggu operasi asinkron tanpa memblokir keseluruhan event loop. Dengan mekanisme ini, server dapat melayani beberapa koneksi pada alamat IP dan port Raspberry Pi.

Setelah membuat berkas kode sumber tersebut, jalankan server Python bernama `main.py` melalui terminal.

Menjalankan server python

```
python main.py
```

2.3.2 Source Code ESP32

Sebelum membuat kode, kita perlu mengetahui alamat IP Raspberry Pi yang telah dikonfigurasi sebagai server sebelumnya. Untuk melihat alamat IP Raspberry Pi, jalankan perintah berikut di terminal:

Melihat IP Address dari Raspberry Pi

```
hostname -I
# Atau Anda bisa menggunakan perintah berikut
ip addr
```

Program di atas akan menampilkan alamat IPv4 dan IPv6 Raspberry Pi. Kita hanya memerlukan alamat IPv4 untuk server WebSocket kita, sebagai contoh `192.168.1.100`.

Informasi
Kita akan menggunakan `WebSockets` library dari Markus Setter
https://github.com/Links2004/arduinoWebSockets

Instal library yang diperlukan terlebih dahulu. Selanjutnya lakukan import library seperti biasa pada ESP32.

Library yang digunakan pada ESP32

```
#include <WiFi.h>             // Untuk koneksi ke internet
#include <WebSocketsClient.h> // Koneksi WebSocket
```

Konfigurasi WiFi dan WebSocket

```
// WiFi
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// WebSocket
const char*    WS_HOST = "192.168.1.100";  // Sesuaikan dengan IP Address Raspberry Pi
const uint16_t WS_PORT = 8765;             // Sesuaikan dengan Port Raspberry Pi

// Untuk membedakan jenis data yang ingin dikirimkan, kita bisa sesuaikan nama path-nya
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000; // Interval pengiriman data

uint32_t counter = 0;
```

Pastikan IP dan Port yang digunakan di kode ESP32 sama dengan yang ada pada Raspberry Pi.

Fungsi Callback WebSocket

```
void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
    switch (type) {
        case WStype_CONNECTED:
            webSocketConnected = true;

            Serial.println("WebSocket connected.");

            // Optional initial message
            webSocket.sendTXT("{\"status\":\"ESP32-S3 connected\"}");
            break;

        case WStype_DISCONNECTED:
            webSocketConnected = false;
            Serial.println("WebSocket disconnected.");
            break;

        case WStype_TEXT:
            Serial.print("Message from server: ");
            Serial.write(payload, length);
            Serial.println();
            break;

        case WStype_ERROR:
            Serial.println("WebSocket error.");
            break;

        default:
            break;
    }
}
```

Pada kode di atas, fungsi callback `void webSocketEvent()` digunakan untuk memproses event WebSocket baik memberi maupun menerima pesan.

Koneksi ke Wi-Fi

```
void connectToWiFi() {
    Serial.print("Connecting to Wi-Fi");

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    // Menunggu koneksi Wi-Fi
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("Wi-Fi connected.");

    Serial.print("ESP32 IP address: ");
    Serial.println(WiFi.localIP());
}
```

Sebelum menggunakan WebSocket, kita perlu menghubungkan ESP32 ke jaringan Wi-Fi. Mode `WIFI_STA` memperbolehkan perangkat untuk menjadi sebuah client yang dapat terhubung ke Wi-Fi Access Point (AP) yang tersedia, mirip seperti telepon seluler yang terhubung ke router Wi-Fi. Koneksi dimulai dengan memanggil fungsi `WiFi.begin()` sesuai dengan nama atau SSID Wi-Fi dan password dari koneksi internet yang kita pakai. Pastikan bahwa internet yang kita hubungkan dengan ESP32 sama dengan yang kita hubungkan dengan Raspberry Pi.

Fungsi Setup pada ESP32

```
void setup() {
    Serial.begin(115200);
    delay(1000);

    connectToWiFi();

    webSocket.begin(WS_HOST, WS_PORT, WS_PATH);  // Memulai koneksi WebSocket
    webSocket.onEvent(webSocketEvent);    // Registrasi fungsi callback yang telah dibuat
    webSocket.setReconnectInterval(5000); // Koneksikan ulang bila terputus selama 5 detik

    // Sambungkan ulang Wi-Fi secara otomatis ketika koneksi terputus
    WiFi.setAutoReconnect(true);
}
```

WebSocket akan dimulai ketika kita berhasil menghubungkan ESP32 ke jaringan. Pada kode di atas, ESP32 akan mencoba terhubung ke Raspberry Pi menggunakan WebSocket sesuai dengan Host, Port, dan Path yang kita atur. Bila koneksi tidak stabil, Wi-Fi dan WebSocket akan mencoba untuk menghubungkan kembali, dengan interval waktu 5 detik untuk WebSocket.

Fungsi Loop pada ESP32

```
void loop() {
    // Memulai loop dari WebSocket
    webSocket.loop();

    unsigned long currentTime = millis();

    if (
        webSocketConnected &&
        currentTime - previousSendTime >= SEND_INTERVAL
    ) {
        previousSendTime = currentTime;

        // Letakkan logika pengiriman akan dijalankan di block kodingan berikut
        // Sebagai contoh, kodingan berikut akan mengirimkan counter tiap detik
        String message = "{\"counter\":";
        message += counter;
        message += "}";

        webSocket.sendTXT(message);

        Serial.print("Sent: ");
        Serial.println(message);

        counter++;
    }
}
```

Metode `webSocket.loop()` harus dipanggil secara berkala untuk memproses event koneksi WebSocket. Setelah interval pengiriman terpenuhi, metode `webSocket.sendTXT()` mengirimkan pesan teks ke server Raspberry Pi.

```
ESP32 device connected
ESP32 message: {"status":"ESP32-S3 connected"}
ESP32 message: {"counter":0}
ESP32 message: {"counter":1}
ESP32 message: {"counter":2}
ESP32 message: {"counter":3}
ESP32 message: {"counter":4}
ESP32 message: {"counter":5}
```

Gambar 2.4: Output pada Server WebSocket

Contoh di atas adalah output yang akan dikeluarkan oleh server python WebSocket yang telah kita buat sebelumnya.

MODUL 3

KONEKTIVITAS MQTT DENGAN MICROPYTHON, RASPBERRY PI PICO W, DAN ECLIPSE MOSQUITTO

TUJUAN

1) Mengetahui protokol MQTT.
2) Memahami proses komunikasi protokol MQTT dengan broker lokal.
3) Memahami implementasi IoT menggunakan Python pada mikrokontroler Raspberry Pi Pico W.

REQUIREMENT

1) Thonny sebagai IDE MicroPython,
2) Raspberry Pi Pico W,
3) Server Lokal (Raspberry Pi 3B).
4) Pemahaman dasar pemrograman Python.

3.1 MQTT

MQTT adalah protokol transport pesan client-server berbasis publish-subscribe yang dirancang ringan untuk lingkungan dengan keterbatasan bandwidth, daya, atau sumber daya perangkat. Client berkomunikasi melalui broker menggunakan topic. MQTT mendukung beberapa tingkat Quality of Service (QoS), sesi, retained message, dan Last Will sesuai versi protokol serta fitur yang digunakan.

3.2 Kelebihan MQTT

MQTT banyak digunakan pada sistem IoT karena beberapa karakteristik berikut:

a. **Ringan dan efisien**

Penggunaan MQTT pada sistem IoT dapat diimplementasikan pada mikrokontroler berdaya rendah karena ukuran data yang kecil, sehingga bandwith jaringan dapat dioptimalkan.

b. **Skalabel**

Arsitektur broker dan pemisahan antara publisher serta subscriber memungkinkan sistem dikembangkan untuk melayani banyak client. Skalabilitas aktual tetap bergantung pada kapasitas broker, desain topic, ukuran pesan, QoS, jaringan, dan pola koneksi.

c. **Andal**

MQTT menyediakan QoS 0, QoS 1, dan QoS 2 dengan jaminan pengiriman yang berbeda. Keandalan tidak sama dengan latensi: QoS yang lebih tinggi dapat menambah pertukaran paket dan overhead. Pemilihan QoS harus disesuaikan dengan risiko kehilangan atau duplikasi pesan.

d. **Didukung secara luas**

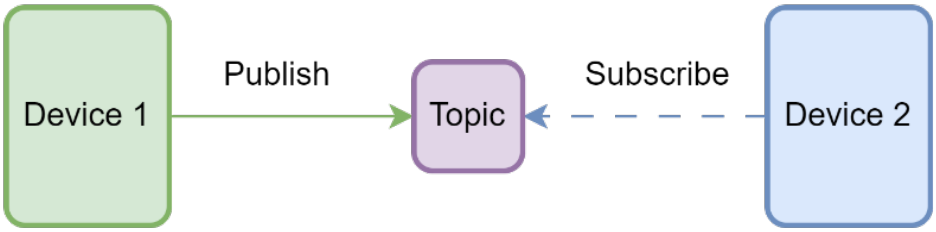
MQTT didukung oleh banyak bahasa pemrograman dan platform, termasuk Python yang memiliki library khusus untuk implementasi protokol MQTT pada perangkat IoT. Oleh karena itu, pengembang dapat mengembangkan perangkat IoT dengan optimal.

3.3 Alur Komunikasi MQTT

Seperti yang telah disebutkan sebelumnya, komunikasi MQTT menerapkan sistem **Publish-Subscribe** untuk melakukan proses transfer data. Pertukaran data antar perangkat yang dilakukan oleh MQTT merupakan **pesan/messages** yang berisi informasi seperti pembacaan sensor atau perintah.

3.3.1 Publish-Subscribe

Istilah publish dan subscribe pada MQTT digunakan saat client bertukar pesan melalui broker. Publisher mengirimkan pesan ke topic tertentu, sedangkan subscriber berlangganan menggunakan topic filter agar broker meneruskan pesan yang sesuai.



Gambar 3.1: Visualisasi Publish - Subscribe

Seperti yang ditunjukkan pada Gambar 3.1, Device 1 memublikasikan pesan ke suatu topic dan Device 2 berlangganan pada topic filter yang sesuai. Broker menerima pesan dari Device 1, kemudian meneruskannya kepada Device 2.

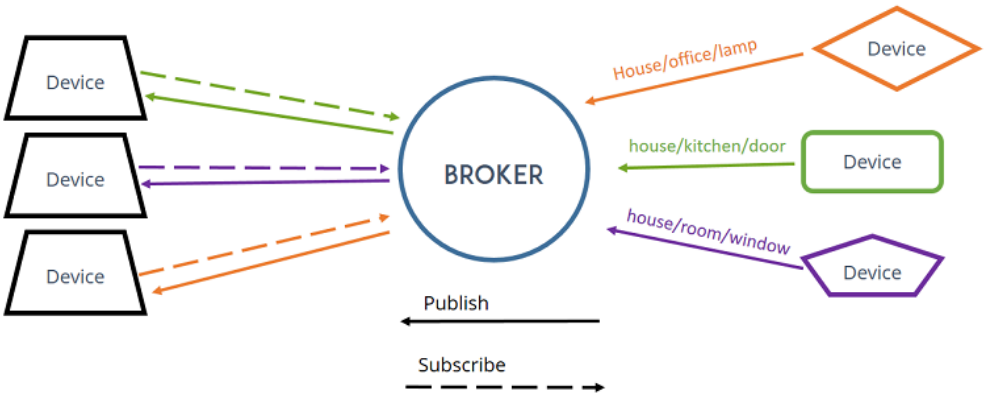
3.3.2 Topics

Topic adalah string hierarkis yang digunakan broker untuk mengarahkan pesan. Client mempublikasikan pesan ke suatu topic, sedangkan subscriber menggunakan topic filter untuk menerima pesan yang sesuai. Topic bersifat case-sensitive; wildcard + dan # dapat digunakan pada subscription sesuai aturan MQTT, tetapi tidak digunakan sebagai nama topic ketika melakukan publish.

Topic MQTT berupa string dan bersifat **case-sensitive**, sehingga perbedaan huruf kapital menghasilkan nama topic yang berbeda.

3.3.3 Broker

Broker menerima pesan dari publisher, mencocokkannya dengan subscription, lalu meneruskannya kepada client yang topic filternya sesuai. Broker juga dapat mengelola sesi, QoS, retained message, autentikasi, dan otorisasi sesuai konfigurasi.



Gambar 3.2: Visualisasi Broker

Terdapat berbagai layanan broker MQTT yang tersedia di internet, misalnya Adafruit IO dan HiveMQ, dengan pilihan paket serta batas penggunaan yang berbeda. Broker juga dapat dijalankan sendiri pada mikrokomputer seperti Raspberry Pi menggunakan software Mosquitto.

3.4 Eclipse Mosquitto

Eclipse Mosquitto adalah message broker MQTT sumber terbuka yang ringan dan dapat dijalankan pada berbagai sistem, termasuk Raspberry Pi. Dengan Mosquitto, broker lokal dapat disiapkan tanpa bergantung pada layanan broker eksternal seperti Adafruit IO dan HiveMQ.



Gambar 3.3: Logo Mosquitto MQTT

3.5 MicroPython

MicroPython adalah implementasi Python yang dioptimalkan untuk berjalan di mikrokontroler dan perangkat berdaya rendah. Diciptakan oleh Damien George, MicroPython memungkinkan pengembang untuk menggunakan bahasa pemrograman Python pada perangkat keras yang memiliki sumber daya terbatas, seperti mikrokontroler, mikroprosesor, dan perangkat Internet of Things (IoT).

Berikut adalah beberapa fitur dan karakteristik utama dari MicroPython:

a. **Ringan dan Optimal**

MicroPython dirancang untuk berjalan pada perangkat keras yang memiliki sumber daya terbatas. Interpreter dan inti bahasa Python diperkecil untuk memastikan kinerja yang optimal pada mikrokontroler dengan kapasitas memori yang rendah.

b. **Dukungan untuk Mikrokontroler Populer**

MicroPython mendukung sejumlah mikrokontroler populer seperti ESP8266, ESP32, STM32, dan sebagainya. Ini memungkinkan pengembang untuk menggunakan bahasa Python pada berbagai perangkat keras yang umum digunakan dalam proyek mikrokontroler.

c. **Interaktif**

Salah satu fitur unik dari MicroPython adalah mode interaktifnya. Pengguna dapat berinteraksi langsung dengan perangkat keras menggunakan REPL (read-eval-print loop) yang mirip dengan Python pada komputer desktop. Hal ini memudahkan pengembangan dan debugging.

d. **Dukungan Modul Standar Python**

Meskipun dibuat untuk mikrokontroler, MicroPython mencakup sejumlah modul standar Python yang dapat ditemui di implementasi Python lainnya. Meskipun modul tersebut dapat berbeda dalam ukuran atau implementasi khusus, banyak modul standar dapat digunakan di MicroPython tanpa modifikasi.

e. **Kemampuan I/O**

MicroPython menyediakan antarmuka untuk mengakses pin I/O dan perangkat keras lainnya pada mikrokontroler, memungkinkan pengembang untuk mengendalikan sensor, motor, dan perangkat keras lainnya.

f. **Dukungan Wi-Fi dan Jaringan**

Beberapa implementasi MicroPython, terutama pada mikrokontroler seperti ESP8266 dan ESP32, menyediakan dukungan untuk konektivitas Wi-Fi dan jaringan, memungkinkan perangkat untuk terhubung dengan internet.

g. **Komunitas dan Ekosistem yang Berkembang**

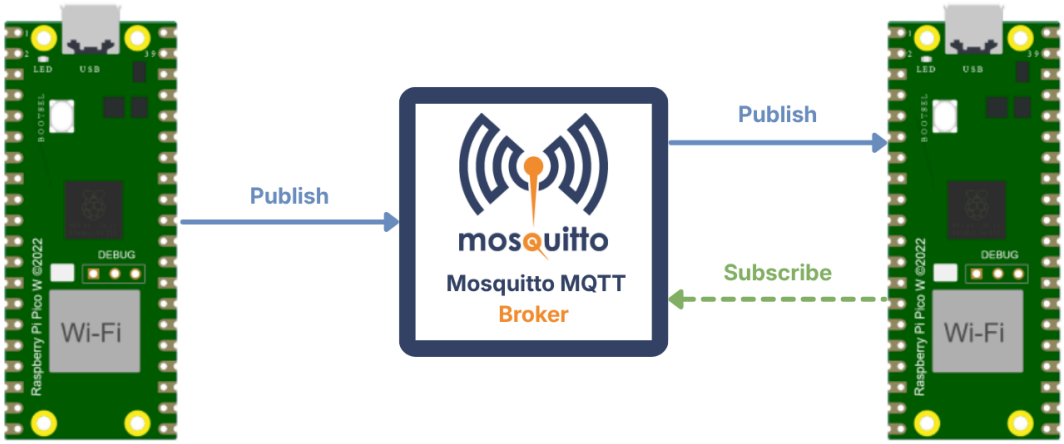
MicroPython memiliki komunitas pengembang yang aktif, dan banyak sumber daya dan tutorial yang tersedia secara online. Ini membuatnya lebih mudah bagi pengembang untuk memulai dengan MicroPython dan memecahkan masalah yang mereka hadapi.

MicroPython memberikan kemudahan pengembangan dan kekuatan bahasa Python pada mikrokontroler, membuatnya menjadi pilihan yang menarik untuk proyek-proyek IoT dan pengembangan perangkat keras berdaya rendah.

3.6 Raspberry Pi Pico W

Raspberry Pi Pico W adalah papan mikrokontroler berbasis RP2040 dengan prosesor dual-core Arm Cortex-M0+, SRAM 264 kB, dan flash eksternal 2 MB pada papan standar. Pico W menambahkan konektivitas Wi-Fi 2,4 GHz IEEE 802.11 b/g/n dan dukungan Bluetooth 5.2 melalui chip nirkabel yang berkomunikasi dengan RP2040. Papan ini dapat diprogram menggunakan MicroPython, C, atau C++; MicroPython bukan satu-satunya bahasa pemrograman yang didukung. Diagram pinout dapat dilihat pada Lampiran 3.

3.7 Implementasi Protokol MQTT



Gambar 3.4: Implementasi MQTT dengan Raspberry Pi Pico W

Berikut alur pengiriman antara Raspberry Pi Pico W dan Mosquitto MQTT:

a. Device 1 memublikasikan pesan ke topic 'A' pada broker MQTT.
b. Device 2 berlangganan pada topic 'A'.
c. Broker MQTT meneruskan pesan dari Device 1 kepada Device 2 sesuai langganannya.

3.7.1 Setup Broker Lokal dengan Raspberry Pi

Untuk menggunakan MQTT, diperlukan broker sebagai perantara antara client yang memublikasikan pesan dan client yang berlangganan. Di sini, komputer papan tunggal Raspberry Pi digunakan untuk menjalankan broker MQTT lokal. Perlu diketahui bahwa bagian ini bersifat opsional bila Anda menggunakan broker eksternal seperti HiveMQ atau Adafruit IO.

Instal paket `mosquitto` dalam Raspberry Pi sebagai berikut di dalam terminal:

Instalasi Mosquitto MQTT Broker

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

Saat menjalankan perintah dengan `sudo`, sistem mungkin meminta kata sandi akun Raspberry Pi. Kita akan menggunakan paket `mosquitto` dan `mosquitto-clients` untuk melakukan setup pada broker MQTT ini.

Setelah itu, jalankan kode berikut di dalam terminal untuk mengaktifkan mosquitto ketika Raspberry Pi dinyalakan:

Mengaktifkan autostart pada Mosquitto MQTT

```
sudo systemctl enable mosquitto.service
```

Untuk melihat alamat IP Raspberry Pi, jalankan perintah berikut di terminal:

Melihat IP Address dari Raspberry Pi

```
hostname -I
# Atau Anda bisa menggunakan perintah berikut
ip addr
```

Program di atas akan menampilkan alamat IPv4 dan IPv6 Raspberry Pi. Kita hanya memerlukan alamat IPv4 untuk broker kita, sebagai contoh `192.168.1.100`.

3.7.1.1 Tanpa Autentikasi

Agar broker dapat diakses dari perangkat lain tanpa autentikasi, kita perlu mengedit konfigurasi pada mosquitto yang akan digunakan. Edit file `mosquitto.conf` yang berada di `/etc/mosquitto/mosquitto.conf` dengan teks editor yang Anda gunakan, contohnya teks editor `nano` di dalam terminal:

Membuka teks editor di dalam terminal

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Ketika pada mode pengeditan, tambahkan beberapa opsi berikut pada bagian bawah baris konfigurasi `mosquitto.conf`:

Konten file konfigurasi dari mosquitto.conf

```
# Konfigurasi Default
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

# Konten baru dari file
listener 1883
allow_anonymous true
```

Pada konfigurasi di atas, konfigurasi `listener 1883` akan mengatur port yang dipakai oleh MQTT, sedangkan `allow_anonymous true` memperbolehkan siapa pun untuk menggunakan broker tersebut tanpa autentikasi. Konfigurasi ini hanya boleh digunakan pada jaringan praktikum yang terisolasi dan tidak boleh diekspos ke internet. Setelah menyimpan konfigurasi, lakukan restart pada service Mosquitto.

Memulai ulang service pada program Mosquitto MQTT

```
sudo systemctl restart mosquitto.service
```

3.7.2 Dengan Autentikasi

Bila Anda ingin menambahkan autentikasi pada broker, diperlukan beberapa konfigurasi tambahan. Buatlah nama pengguna dan kata sandi yang akan digunakan pada broker MQTT.

Menambahkan User dan Password pada program Mosquitto MQTT

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd YOUR_USERNAME
# Setelah menjalankan perintah di atas, masukkan kata sandi saat diminta.

# Ubah permission filenya agar program Mosquitto MQTT dapat membaca filenya

```

```
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

Saat memasukkan kata sandi, terminal biasanya tidak menampilkan karakter yang diketik. Pastikan kata sandi dicatat dan disimpan dengan aman. Setelah itu, ubah konfigurasi default dari `mosquitto.conf` menjadi sebagai berikut:

Konten file konfigurasi dari mosquitto.conf

```
# Konten baru dari file
# Konfigurasi autentikasi berlaku untuk listener yang digunakan

# Konfigurasi Default
pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

# Konten baru dari file
allow_anonymous false
listener 1883
password_file /etc/mosquitto/passwd
```

Berbeda dengan sebelumnya, `allow_anonymous` diatur menjadi `false` agar client tanpa kredensial yang valid tidak dapat menggunakan broker. Autentikasi nama pengguna dan kata sandi tidak mengenkripsi lalu lintas; gunakan TLS apabila broker diakses melalui jaringan yang tidak tepercaya. Berikutnya kita konfigurasi file `passwd` agar program Mosquitto MQTT dapat membaca dengan perintah berikut:

Mengubah permission file dari passwd

```
sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
sudo chmod 600 /etc/mosquitto/passwd
```

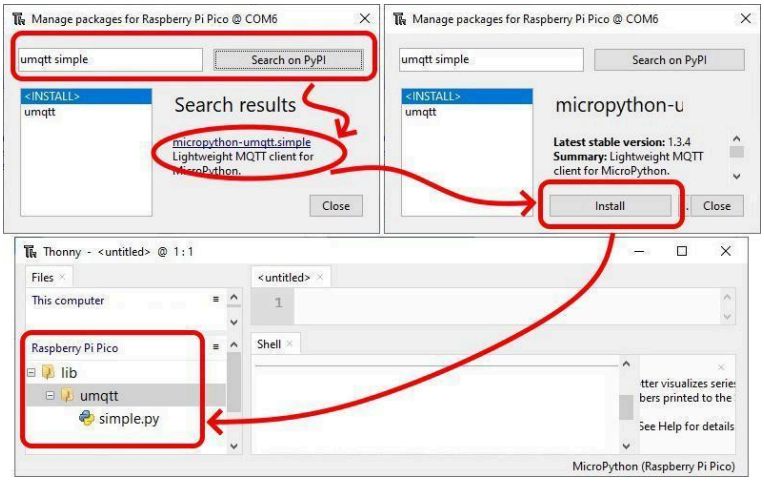
Terakhir, lakukan restart pada service dari program mosquitto.

Memulai ulang service pada program Mosquitto MQTT

```
sudo systemctl restart mosquitto.service
```

3.7.3 Setup Library MQTT pada Raspberry Pi Pico W

Library yang digunakan untuk menjalankan MQTT pada Pico W adalah `micropython-umqtt.simple`. Berikut adalah cara untuk menambahkan library pada Thonny untuk MicroPython:



Gambar 3.5: Tools > Manage Packages

3.7.4 Source Code Raspberry Pi Pico W

3.7.4.1 Publish

a. **Kebutuhan library dan variabel**

Library yang dibutuhkan untuk menggunakan protokol MQTT pada MicroPython adalah library `umqtt.simple`.

MicroPython Library Import

```
import network                      # Untuk mengkoneksikan ke internet
import time                         # Untuk fungsi sleep()
from umqtt.simple import MQTTClient # Library MQTT
import utime                        # Formatting Waktu
```

Setelah import library, maka langkah selanjutnya yang akan dilakukan adalah mendeklarasikan variabel yang akan digunakan. Variabel yang digunakan antara lain:

- Variabel untuk jaringan Wi-Fi
- Variabel untuk MQTT client
- Variabel yang akan dipublikasikan ke Broker

Source code untuk inisialisasi variabel

```
# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, Adafruit IO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-publish
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Angka yang akan dipublish
counter = 0
```

b. **Menghubungkan Pico W ke Wi-Fi dan MQTT server**

Setelah variabel dideklarasikan, maka langkah selanjutnya adalah dengan menghubungkan Pico W ke Wi-Fi dan MQTT server. Fungsi yang digunakan untuk terhubung ke Wi-Fi adalah:

```
wlan.connect("<YOUR WIFI SSID>", "<YOUR WIFI PASSWORD>")
```

dan fungsi untuk terhubung ke MQTT client adalah:

```
mqtt_client.connect()
```

Source Code untuk terhubung ke Wi-Fi dan MQTT server

```
# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()
```

c. **Kode block try**

Pada kode di bawah ini, maka Pico W akan melakukan publish informasi dari variabel `count` menggunakan fungsi:

```
mqtt_client.publish("<Topic yang dituju>", str(<Nilai yang akan dipublikasikan>))
```

Source Code untuk block try (seperti fungsi loop pada Arduino)

```
try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')
```

```
23
24  # Ketika program python dimatikan
25  finally:
26    mqtt_client.disconnect()
```

Pengiriman/publish informasi ke server MQTT tersebut akan berulang setiap 3 detik menggunakan fungsi `time.sleep(3)`.

3.7.4.2 Subscribe

Sama seperti sebelumnya, library yang dibutuhkan untuk melakukan subscribe pada mqtt di MicroPython adalah library `umqtt.simple`

MicroPython Library Import

```
1  import network
2  import time
3  from umqtt.simple import MQTTClient
4  import utime
```

Source code untuk inisialisasi variabel

```
1  # Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
2  wifi_ssid     = "<YOUR SSID WIFI>"
3  wifi_password = "<YOUR WIFI PASSWORD>"
4
5  # Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, Adafruit IO)
6  # dan lain-lain
7  mqtt_host     = "<MQTT HOST ADDRESS>"
8
9  # (Opsional) Bila menggunakan autentikasi
10 mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
11 mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"
12
13 # Isi sesuai dengan topik yang ingin di-subscribe
14 mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"
15
16 # Isi ID client random (harus unik tiap client)
17 mqtt_client_id = "iij0dfq8f012nif2jnfijn"
```

Source Code untuk terhubung ke Wi-Fi dan MQTT server

```
1  # Koneksi ke WiFi
2  wlan = network.WLAN(network.STA_IF)
3  wlan.active(True)
4  wlan.connect(wifi_ssid, wifi_password)
5
6  while wlan.isconnected() == False:
7    print('Waiting for connection...')
8    time.sleep(1)
9
10 print("Connected to WiFi")
11
12 # Initialize our MQTTClient and connect to the MQTT server
13 mqtt_client = MQTTClient(
14   client_id=mqtt_client_id,
15   server=mqtt_host,
16   # Bila menggunakan autentikasi
17   user=mqtt_username,
18   password=mqtt_password
19 )
```

Perbedaan kode sumber subscribe dan publish terletak pada fungsi callback yang akan dipanggil setiap ada informasi baru yang datang di Topic yang dilanggani.

Fungsi Callback MQTT ketika ada pesan baru

```
def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
    current_time[3], current_time[4], current_time[5]
  )
  # Debug print out of what was received over MQTT
  print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()
```

Setelah membuat fungsi callback, langkah selanjutnya adalah melakukan subscribe ke Topic yang Anda inginkan menggunakan fungsi:

```
mqtt_client.subscribe("<Topic_yang_dilanggani>")
```

Melakukan Subscribe pada topik yang dipilih

```
# Once connected, subscribe to the MQTT topic
mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")
```

Source Code untuk block try (seperti fungsi loop pada Arduino)

```
try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

Pada kode di atas, `mqtt_client.wait_msg()` akan menunggu pesan dari client MQTT yang memublikasikan pesan ke topik tersebut. Bila mendapatkan data, maka akan menghasilkan output sesuai dengan tindakan yang kita sudah definisikan pada fungsi `mqtt_subscription_callback()`.

MODUL 4

KOMUNIKASI LONG RANGE (LoRa)

TUJUAN

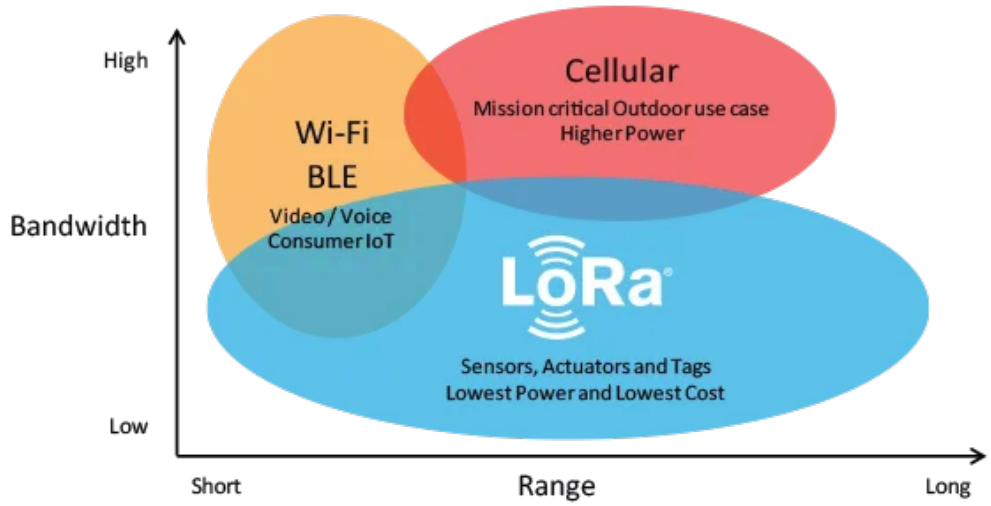
1) Mengetahui teknologi modulasi Long Range (LoRa).
2) Memahami komunikasi menggunakan LoRa, terutama LoRaWAN.

REQUIREMENT

1) Arduino IDE/PlatformIO,
2) LoRa Module/Cosmic LoRa Aurora V2 ESP32 LoRa Module,
3) Akun ANTARES,
4) Sensor (opsional),
5) Actuator (opsional)

4.1 LoRa

LoRa (Long Range) adalah teknik modulasi radio spread spectrum untuk komunikasi jarak jauh dengan laju data rendah dan konsumsi daya yang relatif rendah. LoRa bekerja pada pita frekuensi sub-GHz yang ketentuannya berbeda menurut wilayah. Teknologi ini sesuai untuk pengiriman paket data kecil pada aplikasi seperti pertanian pintar, kota pintar, pemantauan industri, dan penginderaan lingkungan. LoRa bukan protokol jaringan; protokol jaringan yang umum dibangun di atas LoRa adalah LoRaWAN.



Gambar 4.1: LoRa Bandwidth - Range

LoRa menggunakan Chirp Spread Spectrum (CSS) dan teknik koreksi kesalahan untuk meningkatkan ketahanan penerimaan sinyal. Parameter seperti spreading factor, bandwidth, coding rate, frekuensi, dan daya pancar memengaruhi jangkauan, airtime, ketahanan gangguan, serta laju data. Topologi star-of-stars merupakan karakteristik arsitektur LoRaWAN, bukan sifat wajib komunikasi LoRa peer-to-peer.

4.2 Alur Komunikasi LoRa

Komunikasi LoRa menggunakan pita frekuensi yang mengikuti ketentuan wilayah dan regulasi setempat. Contohnya, LoRaWAN menggunakan rencana regional EU863-870 di sebagian Eropa dan US902-928 di Amerika Serikat. Untuk Indonesia, parameter regional yang digunakan adalah AS923-2 pada pita **920–923 MHz**. Pengguna harus memastikan frekuensi, daya pancar, duty cycle, dan sertifikasi perangkat sesuai regulasi yang berlaku.

Pilihan frekuensi memengaruhi jangkauan dan kinerja komunikasi. Perangkat akhir, seperti sensor IoT atau objek yang terhubung, menggunakan modulasi LoRa untuk mengubah data yang dikodekan menjadi sinyal chirp, yang ditransmisikan melalui gelombang radio. LoRa dapat mengirim data dengan menggunakan teknik modulasi yang memungkinkan transmisi informasi secara efisien dalam jarak jauh sekaligus menghemat daya.

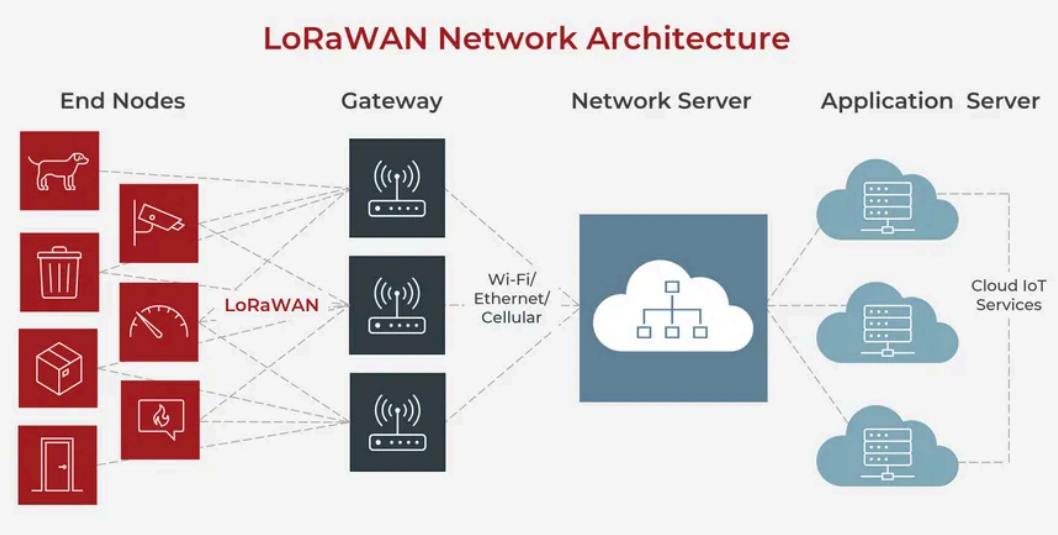
LoRa menggunakan modulasi **Chirp Spread Spectrum (CSS)**, yaitu sinyal chirp yang frekuensinya berubah terhadap waktu. Pada komunikasi peer-to-peer, transmitter dan receiver berkomunikasi langsung apabila menggunakan parameter radio yang kompatibel, seperti frekuensi, bandwidth, spreading factor, coding rate, dan sync word. Pada jaringan LoRaWAN, paket dari end-device dapat diterima oleh satu atau beberapa gateway, kemudian diteruskan melalui jaringan IP ke network server. Dengan demikian, keberadaan gateway dan server merupakan bagian dari LoRaWAN, bukan persyaratan untuk seluruh komunikasi LoRa.

4.3 LoRaWAN

LoRaWAN (Long Range Wide Area Network) merupakan protokol komunikasi jaringan yang dibangun di atas teknologi modulasi LoRa. Jika LoRa mengatur bagaimana sinyal radio dikirimkan melalui udara, LoRaWAN mengatur bagaimana perangkat bergabung ke dalam jaringan, mengirim dan menerima data, melakukan autentikasi, menjaga keamanan data, serta meneruskan data dari perangkat menuju aplikasi pengguna.

Secara umum, LoRaWAN melibatkan empat komponen utama, yaitu:

a. **End-device**, berupa sensor, aktuator, atau perangkat IoT yang mengumpulkan dan mengirimkan data.
b. **Gateway**, yang menerima paket LoRa dari berbagai **end-device** dan meneruskannya melalui jaringan IP.
c. **Network server**, yang mengelola perangkat, memeriksa keamanan paket, menghilangkan data duplikat, dan menentukan jalur pengiriman data.
d. **Application server**, yang mengolah dan menampilkan data agar dapat digunakan oleh pengguna atau sistem aplikasi.



Gambar 4.2: Arsitektur LoRaWAN

LoRaWAN menggunakan topologi **star-of-stars**. Pada topologi ini, **end-device** tidak harus terhubung dengan satu gateway tertentu. Satu paket data dapat diterima oleh beberapa gateway sekaligus, kemudian diteruskan ke **network server**. Server akan memilih dan memproses paket yang sesuai serta menghapus paket duplikat. Pendekatan ini dapat meningkatkan jangkauan dan keandalan komunikasi tanpa mengharuskan perangkat akhir melakukan proses **routing** seperti pada jaringan **mesh**.

Penggunaan LoRaWAN sesuai untuk aplikasi yang hanya mengirimkan data berukuran kecil secara berkala atau berdasarkan suatu kejadian, seperti data suhu, kelembapan, kualitas udara, posisi aset, ketinggian air, status meter, kondisi mesin, dan status perangkat. LoRaWAN dipilih karena mampu menyediakan jangkauan komunikasi yang luas, konsumsi daya rendah, kapasitas perangkat yang besar, serta biaya pembangunan jaringan yang relatif rendah.

Alur komunikasi LoRaWAN dimulai ketika **end-device** membaca data dari sensor, kemudian mengemas dan mengenkripsi data tersebut sesuai format LoRaWAN. Data dikirimkan menggunakan modulasi LoRa dan dapat diterima oleh satu atau beberapa gateway. Gateway meneruskan paket melalui koneksi internet atau jaringan lokal menuju **network server**. Setelah

dilakukan autentikasi, pemeriksaan integritas, penghapusan paket duplikat, dan pengelolaan jaringan, data diteruskan ke **application server** untuk disimpan, dianalisis, atau ditampilkan pada dashboard. Untuk komunikasi **downlink**, data dikirimkan kembali dari **application server** menuju **end-device** melalui **network server** dan gateway.

LoRaWAN sebaiknya digunakan ketika suatu sistem memiliki karakteristik berikut:

a. Membutuhkan jangkauan komunikasi hingga beberapa kilometer;
b. Menggunakan perangkat bertenaga baterai yang harus bertahan lama;
c. Memiliki banyak sensor yang tersebar di wilayah luas;
d. Hanya mengirimkan data dalam ukuran kecil dengan interval tertentu;
e. Tidak membutuhkan kecepatan data tinggi atau komunikasi waktu nyata;
f. Membutuhkan keamanan, autentikasi perangkat, dan pengelolaan jaringan secara terpusat.

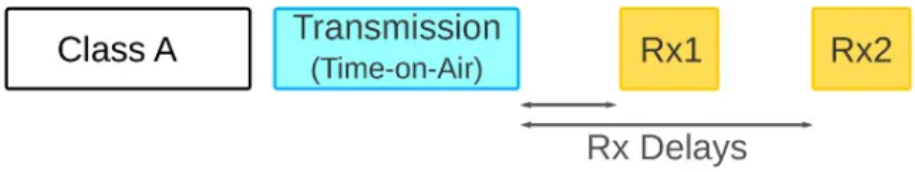
Sebaliknya, LoRaWAN kurang sesuai untuk aplikasi yang membutuhkan pengiriman video, audio, file berukuran besar, kontrol dengan latensi sangat rendah, atau komunikasi data secara terus-menerus. Untuk kebutuhan tersebut, teknologi seperti Wi-Fi, Ethernet, jaringan seluler, atau protokol komunikasi lain dengan kapasitas data lebih tinggi lebih sesuai.

4.4 Tipe Kelas LoRaWAN

Kelas perangkat LoRaWAN terdiri atas Kelas A, Kelas B, dan Kelas C. Kelas tersebut mengatur waktu perangkat membuka jendela penerimaan downlink dan berdampak pada latensi, konsumsi daya, serta pola komunikasi. Semua end-device LoRaWAN wajib mendukung Kelas A; dukungan Kelas B dan Kelas C bersifat tambahan.

4.4.1 Class A

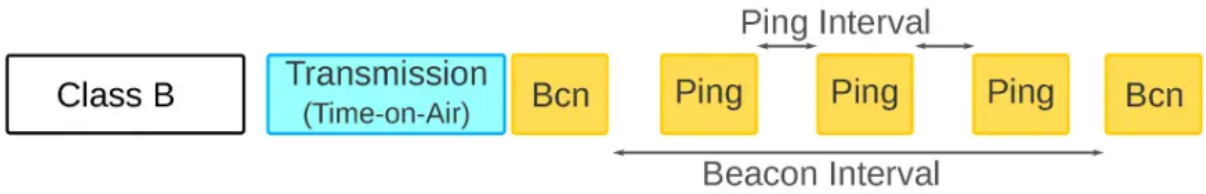
Pada Kelas A, komunikasi uplink dimulai oleh end-device. Setelah setiap uplink, perangkat membuka dua jendela penerimaan downlink. Pola ini memberikan konsumsi daya paling rendah, tetapi downlink harus menunggu hingga perangkat melakukan uplink.



Gambar 4.3: Komunikasi LoRa Class A

4.4.2 Class B

Kelas B menambahkan beacon jaringan dan ping slot terjadwal di samping mekanisme Kelas A. Dengan jadwal tersebut, server memiliki peluang downlink yang lebih dapat diprediksi, tetapi konsumsi daya dan kompleksitas sinkronisasi lebih tinggi daripada Kelas A.



Gambar 4.4: Komunikasi LoRa Class B

4.4.3 Class C

Kelas C menjaga penerima tetap terbuka hampir sepanjang waktu, kecuali ketika perangkat sedang mengirim. Latensi downlink menjadi paling rendah, tetapi konsumsi daya paling tinggi sehingga umumnya digunakan pada perangkat dengan sumber daya yang stabil.



Gambar 4.5: Komunikasi LoRa Class C

4.5 ANTARES

ANTARES adalah platform horizontal Internet of Things yang dikembangkan oleh PT Telkom Indonesia TBK. Platform ini menyediakan layanan untuk mengatur dan mengkoneksikan perangkat IoT, menyimpan data sensor, dan menyajikan data melalui dashboard ataupun aplikasi lainnya. ANTARES melayani berbagai implementasi IoT, termasuk smart homes, smart metering, tracking asset, dan smart buildings.

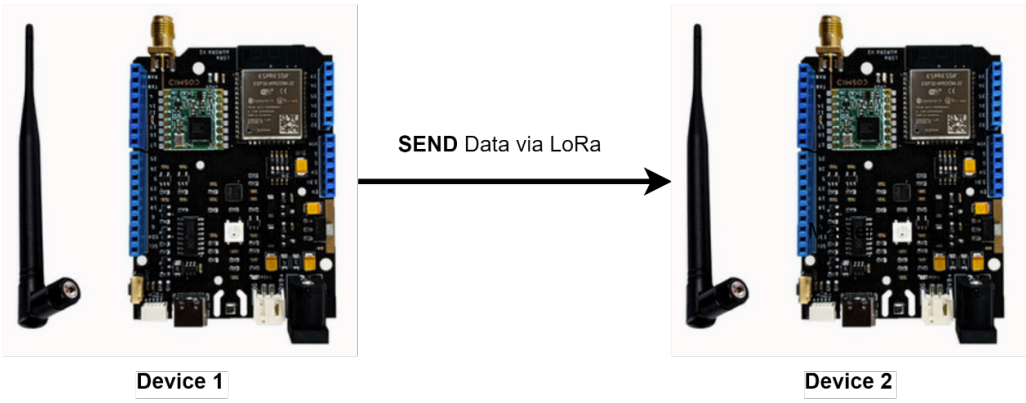


Gambar 4.6: ANTARES Logo

ANTARES memiliki dua layanan utama: IoT platform dan IoT connectivity. Layanan connectivity melayani teknologi seperti LoRaWAN, NB-IoT, dan GPRS, memperbolehkan perangkat IoT untuk mengirimkan data ke platform ANTARES untuk monitoring dan pemrosesan. ANTARES merupakan IoT Platform yang akan kita pakai di implementasi pada bagian berikutnya.

4.6 Implementasi Protokol LoRa

4.6.1 Peer-to-peer LoRa



Gambar 4.7: Visualisasi P2P LoRa

Implementasi peer-to-peer ini menggunakan modulasi LoRa dengan alur seperti pada gambar di atas.

4.6.1.1 Cosmic LoRa Aurora V2

Cosmic LoRa Aurora V2 merupakan dev board LoRa yang menggunakan transceiver LoRa RFM95W yang dikeluarkan oleh Cosmic dan dapat dilihat pada Lampiran 4. Board ini memakai ESP32 sebagai mikrokontrolernya sehingga selain memiliki LoRa, board ini juga sudah dapat menggunakan konektivitas Wi-Fi. Selain itu, LoRa Aurora V2 ini memiliki 3 sumber tenaga yang dapat digunakan, yaitu USB, LiPo Battery, dan DC Jack dan sudah memiliki sistem smart power switching sehingga secara otomatis dapat memilih sumber tenaga yang digunakan, seperti jika LiPo Battery sedang diisi, maka sistem dapat secara otomatis berganti ke USB atau DC Jack.

4.6.1.2 LoRa Library by Sandeep Mistry

Library LoRa yang dikembangkan oleh Sandeep Mistry ini adalah library yang akan digunakan pada implementasi LoRa menggunakan Cosmic LoRa Aurora V2. Library ini merupakan library yang digunakan untuk mengirim dan menerima data menggunakan komunikasi LoRa dengan header `LoRa.h`.

Information
Library yang digunakan berasal dari repositori berikut:
https://github.com/sandeepmistry/arduino-lora

4.6.1.3 Source Code LoRa Peer-to-Peer

Pada kode sumber LoRa peer-to-peer baik untuk transmitter dan receiver menggunakan contoh kode dari Cosmic yang tersedia pada laman web berikut:

Source Code
https://github.com/cosmic-id/cosmic-lora-aurora/blob/main/examples/aurora-v2-lora-p2p-receiver/aurora-v2-lora-p2p-receiver.ino

- **Transmitter**

a. **Inisialisasi**

Inisialisasi Transmitter LoRa

```
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;
```

Dalam menginisialisasi P2P LoRa, kita akan menggunakan library `SPI.h` dan `LoRa.h` oleh Sandeep Mistry. Setelah itu tentukan pin-pin LoRa mana yang digunakan di setiap `#define`. Pada kode di atas ada variabel tambahan yang diinisialisasi, yaitu variabel `counter` yang akan digunakan sebagai nilai yang akan dikirim ke receiver.

b. **Setup**

Setup Transmitter LoRa

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

Pada bagian `setup()` untuk LoRa sebagai transmitter, ada beberapa hal yang harus kita tentukan:

- **Deklarasi pinMode** untuk pin Enable (Berdasarkan kode, maka pin enable terdapat pada konstanta `LORA_EN` dan mengubah state-nya menjadi `HIGH`.
- **Setup pin** untuk modul LoRa menggunakan fungsi `LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN)`.
- **Memulai modul LoRa** menggunakan fungsi `LoRa.begin(frequency)` dengan parameter `frequency` diisi dengan frekuensi yang sesuai regulasi wilayah. Untuk contoh praktikum di Indonesia digunakan `920E6` pada pita 920–923 MHz (AS923-2). Pastikan frekuensi dan daya pancar sesuai regulasi serta spesifikasi perangkat.
- **Menentukan sync word** yang disesuaikan antara transmitter dan receiver. Sync word merupakan parameter sinkronisasi, bukan alamat perangkat atau mekanisme keamanan. Nilainya berada pada rentang 0x00–0xFF dan ditentukan menggunakan fungsi `LoRa.setSyncWord(sync_word)`.

c. **Loop**

Loop Transmitter LoRa

```
void loop() {
  Serial.print("Sending packet: ");
  Serial.println(counter);

  //Send LoRa packet to receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
```

```
  LoRa.print(counter);
  LoRa.endPacket();

  counter++;
  delay(5000);
}
```

- **Receiver**

a. **Inisialisasi**

Inisialisasi Receiver LoRa

```
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

```

Tidak ada perbedaan antara inisialisasi untuk transmitter dan receiver, seperti pin dan library yang digunakan. Namun, pada kode di atas tidak ada tambahan variabel `counter` seperti pada transmitter yang digunakan sebagai data yang akan dikirim melalui LoRa.

b. **Setup**

Setup Receiver LoRa

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

Pada bagian `setup()` di sisi receiver, perlu diperhatikan pada bagian frekuensi dan sync word yang digunakan. Kedua hal tersebut harus memiliki nilai yang sama baik dari sisi pengirim (transmitter) dan penerima (receiver). Pada kode sumber kali ini, frekuensi yang digunakan adalah 920E6 dan untuk sync word menggunakan nilai 0xF3.

c. **Loop**

Loop Receiver LoRa

```
void loop() {
  // Mencoba untuk parse paket yang diterima
  int packetSize = LoRa.parsePacket();

  if (packetSize) {
    // Menerima paket
    Serial.print("Received packet '");

    // Membaca paket yang diterima
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Menampilkan RSSI/Sinyal dari paket yang diterima
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

Pada bagian `loop()`, receiver akan menerima data dari transceiver yang memiliki sync word yang sama. `LoRa.parsePacket()` akan memeriksa paket yang datang. Jika ada paket yang datang, maka paket akan dibaca menggunakan `LoRa.readString()` selama paket tersedia sesuai dengan kondisi `LoRa.available()`. Pada bagian terakhir, `LoRa.packetRssi()` akan mendapatkan RSSI atau kekuatan sinyal dari paket LoRa yang didapat dalam satuan dBm.

4.6.2 Implementasi LoRaWAN Menggunakan ANTARES

Proses komunikasi dapat dijelaskan sebagai berikut:

End Device → LoRaWAN Gateway → Platform ANTARES

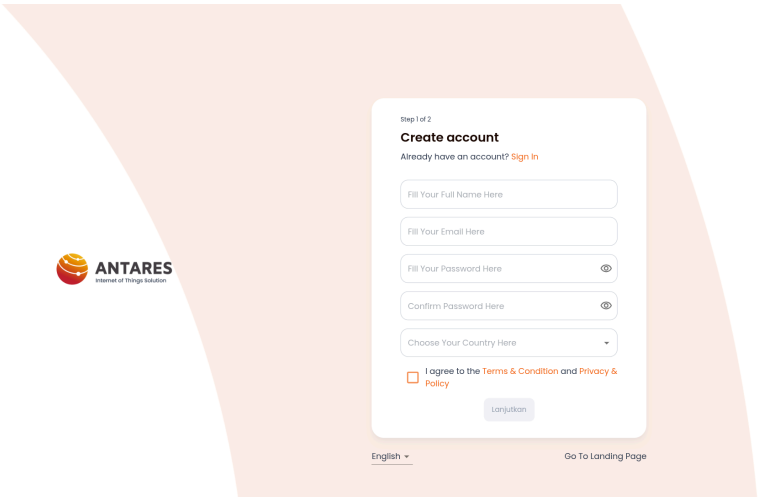
End device mengirimkan data menggunakan modulasi LoRa dan protokol LoRaWAN. Gateway menerima paket radio dan meneruskannya ke infrastruktur ANTARES melalui koneksi Internet.

Implementasi ini menggunakan LoRaWAN Class A dan Activation by Personalization atau ABP. Pada aktivasi ABP, perangkat menggunakan alamat perangkat dan kunci sesi yang telah ditentukan sebelumnya tanpa melakukan prosedur join setiap kali perangkat dinyalakan.

4.6.2.1 Membuat Application dan Device di ANTARES

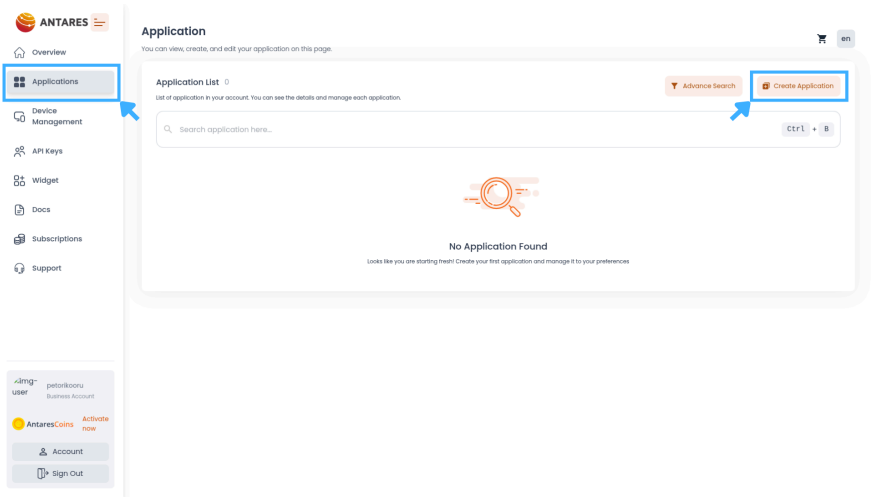
Sebelum mengunggah program, buat sebuah application dan device pada konsol ANTARES. Application dan device dibuat melalui langkah-langkah berikut:

a. Masuk ke konsol ANTARES.



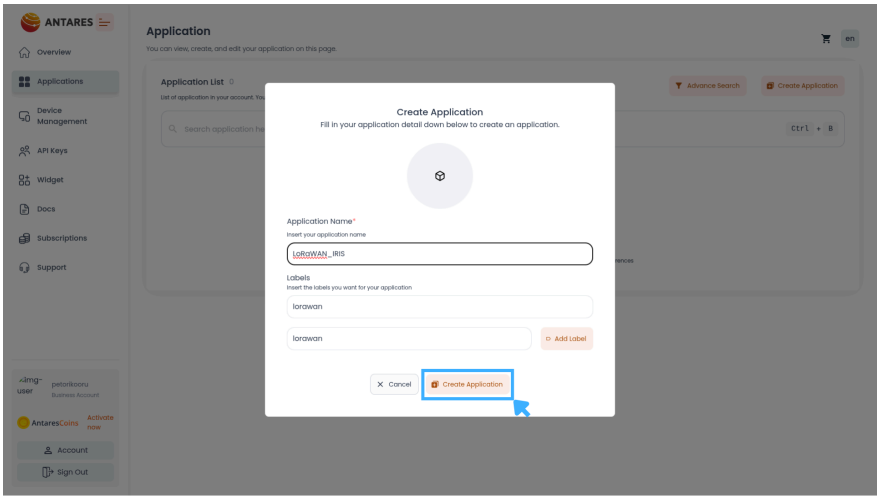
Gambar 4.8: Pendaftaran ANTARES

b. Buka halaman **Application**.



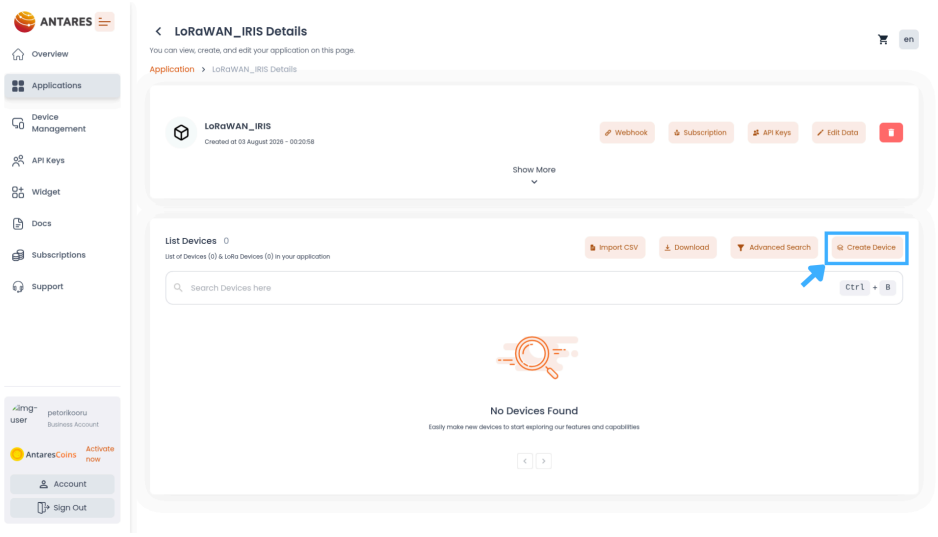
Gambar 4.9: Dashboard Application ANTARES

c. Buat application baru dengan tombol **Create Application**.

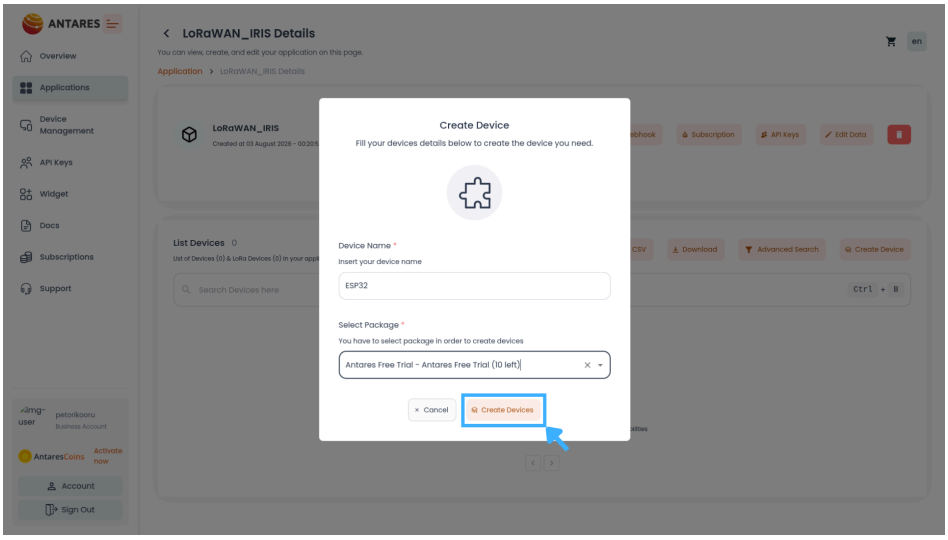


Gambar 4.10: Dialog Pembuatan Application ANTARES

d. Selanjutnya, halaman akan dialihkan ke halaman device. Buat device baru dengan tombol **Create Device**.

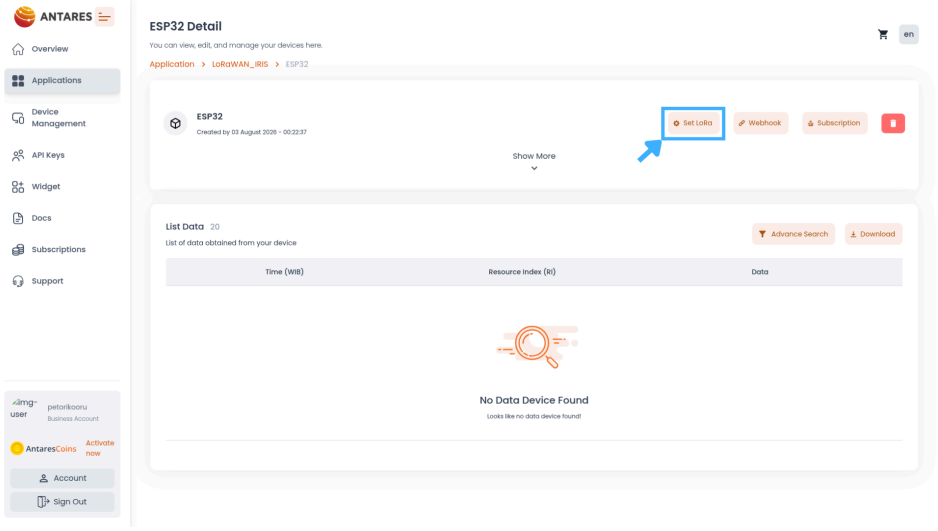


Gambar 4.11: Halaman Device pada Application ANTARES



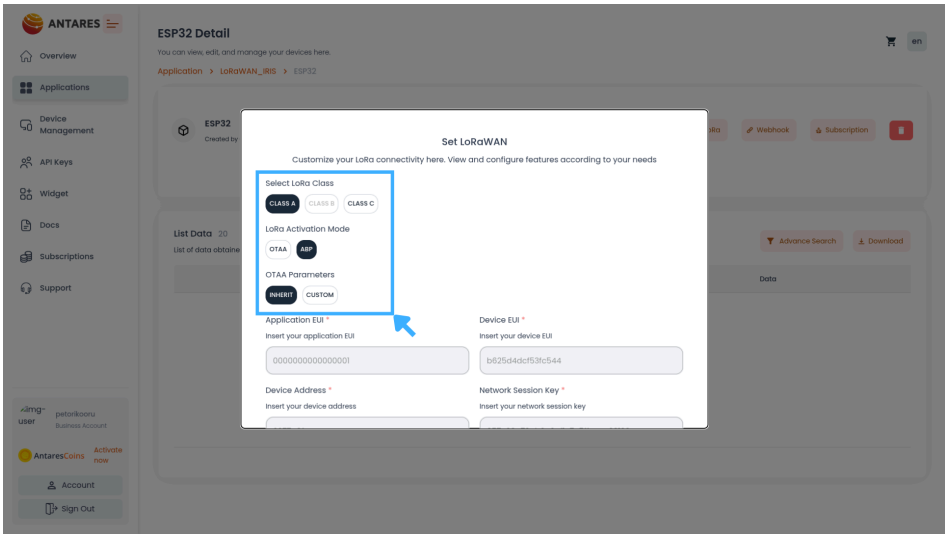
Gambar 4.12: Dialog Pembuatan Device ANTARES

e. Setelah itu, halaman akan dialihkan ke detail device. Pilih **Set LoRa**.



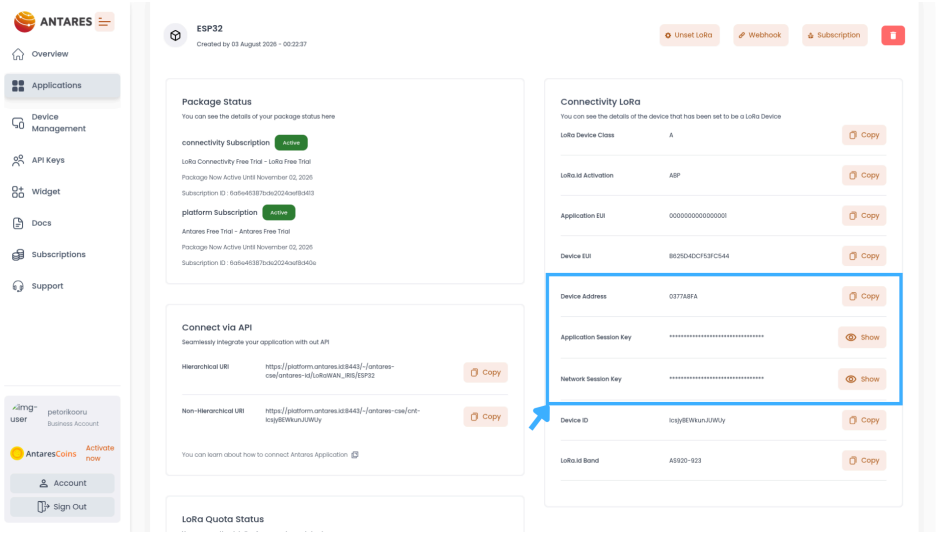
Gambar 4.13: Detail Device ANTARES

f. Konfigurasikan pengaturan LoRa seperti berikut:



Gambar 4.14: Detail Device ANTARES

g. Terakhir, simpan parameter LoRaWAN yang dihasilkan. Parameter yang diperlukan dapat dilihat pada gambar berikut.



Gambar 4.15: Parameter LoRaWAN Device ANTARES

ANTARES menghasilkan parameter ABP berikut:

- **Device Address** atau `DevAddr`;
- **Network Session Key** atau `NwkSKey`;
- **Application Session Key** atau `AppSKey`.

Salin dan simpan parameter tersebut sebelum menyelesaikan konfigurasi device. Parameter tersebut harus dimasukkan ke dalam program Arduino.

Information
Network Session Key dan Application Session Key merupakan identitas rahasia. Jangan memublikasikan kunci yang sebenarnya dalam laporan ataupun repositori publik.

4.6.2.2 Memasang Library LoRaWAN ANTARES

Pasang library LoRaWAN ANTARES bernama `AntaresLoraID` pada Arduino IDE.

Information
Library tersedia melalui dokumentasi berikut:
https://docs.antares.id/pendahuluan/instalasi-library-arduino/antares-lorawan

Program menggunakan file header `lorawan.h` yang disediakan oleh library LoRaWAN ANTARES.

4.6.2.3 Program Source Code LoRaWAN

Program berikut mengonfigurasi Cosmic LoRa Aurora V2 sebagai end device LoRaWAN Class A menggunakan Activation by Personalization atau ABP. Perangkat mengirimkan data counter ke ANTARES setiap 60 detik dan memeriksa pesan downlink setelah setiap transmisi uplink.

Program diawali dengan menyertakan library SPI dan LoRaWAN ANTARES:

Library yang Diperlukan

```
#include <SPI.h>
#include <lorawan.h>
```

Library `SPI.h` mengelola komunikasi antara ESP32 dan modul radio RFM95W. Library `lorawan.h` menyediakan fungsi-fungsi yang diperlukan untuk mengonfigurasi dan mengoperasikan koneksi LoRaWAN.

Definisi berikut menentukan konfigurasi pin RFM95W yang digunakan oleh Cosmic LoRa Aurora V2:

Konfigurasi Pin Radio LoRa

```
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18
```

GPIO 15 digunakan sebagai pin chip-select atau NSS. GPIO 0 terhubung ke pin reset, sedangkan GPIO 27 dan GPIO 2 terhubung ke DIO0 dan DIO1. GPIO 32 mengendalikan rangkaian pengaktif radio dan harus diatur menjadi `HIGH` sebelum radio diinisialisasi.

Kredensial ABP yang dihasilkan oleh ANTARES disimpan dalam variabel berikut:

Parameter ABP ANTARES

```
const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";
```

Variabel `devAddr` berisi alamat perangkat LoRaWAN. Variabel `nwkSKey` berisi Network Session Key, sedangkan `appSKey` berisi Application Session Key. Ganti nilai placeholder dengan parameter yang dihasilkan melalui konsol ANTARES.

Variabel berikut menentukan interval transmisi dan menyimpan status program:

Variabel Program

```
const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;
```

Nilai `interval` dinyatakan dalam milidetik. Nilai `60000` menyebabkan perangkat mengirimkan satu uplink setiap 60 detik. Variabel `previousMillis` menyimpan waktu transmisi sebelumnya, sedangkan `messageCounter` mencatat jumlah pesan yang dihasilkan oleh perangkat. Array `message` menyimpan pesan uplink. Array `receivedData` menyimpan data downlink yang diterima dari ANTARES.

Pemetaan pin RFM95W kemudian diteruskan ke library LoRaWAN:

Pemetaan Pin RFM95W

```
const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
  .DIO0 = LORA_DIO0,
  .DIO1 = LORA_DIO1,
};
```

Struktur `RFM_pins` memungkinkan library berkomunikasi dengan radio RFM95W menggunakan pin ESP32 yang sesuai.

Fungsi `setup()` menginisialisasi Serial Monitor, rangkaian pengaktif radio, antarmuka SPI, modul RFM95W, dan parameter LoRaWAN:

Fungsi Setup

```
void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz:
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);
```

```
28
29    // It will be available to multiple LoRaWAN channels
30    lora.setChannel(MULTI);
31
32    // The transmission power should comply with the applicable regional frequency
33    // regulations and the requirements of the LoRaWAN network
34    lora.setTxPower1(15);
35
36    // ABP credentials
37    // Because ABP uses predefined session parameters,
38    // the device does not perform an OTAA join procedure before transmitting data.
39    lora.setNwkSKey(nwkSKey);
40    lora.setAppSKey(appSKey);
41    lora.setDevAddr(devAddr);
42
43    Serial.println("LoRaWAN Class A ABP configuration completed");
44  }
```

Fungsi `loop()` mengendalikan transmisi uplink berkala dan penerimaan downlink:

Loop Utama

```
1   void loop() {
2     unsigned long currentMillis = millis();
3
4     if (currentMillis - previousMillis >= interval) {
5       previousMillis = currentMillis;
6
7       snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);
8
9       Serial.print("Sending: ");
10      Serial.println(message);
11
12      lora.sendUplink(message, strlen(message), 0, 5);
13
14      messageCounter++;
15
16      channel = lora.getChannel();
17
18      Serial.print("Channel: ");
19      Serial.println(channel);
20    }
21
22    lora.update();
23
24    receiveStatus = lora.readData(receivedData);
25
26    if (receiveStatus > 0) {
27      bool isText = true;
28
29      for (int i = 0; i < receiveStatus; i++) {
30        if (!(
31          (receivedData[i] >= 32 && receivedData[i] <= 126) ||
32           receivedData[i] == 10 || receivedData[i] == 13)
33          ){
34          isText = false;
35          break;
36        }
37      }
38
39      if (isText) {
40        Serial.print("Received text: ");
41
42        for (int i = 0; i < receiveStatus; i++) {
43          Serial.print(receivedData[i]);
44        }
```

```
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>( receivedData[i]) < 0x10) {
          Serial.print('0');
        }

        Serial.print(static_cast<byte>( receivedData[i]), HEX);
        Serial.print(' ');
      }
    }

    Serial.println();
  }
}
```

Pesan uplink dibuat menggunakan nilai message counter saat ini:

Membuat Pesan Uplink

```
snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);
```

Sebagai contoh, tiga pesan pertama yang dihasilkan adalah:

Pesan yang Dihasilkan

```
LoRa Counter-0
LoRa Counter-1
LoRa Counter-2
```

Pesan dikirimkan menggunakan fungsi `sendUplink()`:

Mengirim Data ke ANTARES

```
lora.sendUplink(message, strlen(message), 0, 5);
```

Parameter pertama berisi buffer pesan. Parameter kedua berisi jumlah byte yang akan dikirimkan. Parameter ketiga menentukan apakah uplink bersifat confirmed. Nilai `0` memilih unconfirmed uplink. Parameter keempat menentukan LoRaWAN application port 5. Setelah mengirimkan pesan, program meningkatkan message counter dan membaca kanal yang digunakan untuk transmisi:

Informasi Counter dan Kanal

```
messageCounter++;

channel = lora.getChannel();

Serial.print("Channel: ");
Serial.println(channel);
```

Program kemudian memeriksa apakah data downlink telah diterima:

Membaca Data Downlink

```
receiveStatus = lora.readData(receivedData);
```

Nilai kembalian menunjukkan jumlah byte yang diterima. Jika nilainya lebih besar dari nol, program memeriksa apakah setiap byte merepresentasikan karakter yang dapat dicetak.

Apabila seluruh byte yang diterima dapat dicetak, downlink ditampilkan sebagai teks:

Menampilkan Downlink Teks

```
Serial.print("Received text: ");

for (int i = 0; i < receiveStatus; i++) {
  Serial.print(receivedData[i]);
}
```

Sebagai contoh, downlink ANTARES yang memuat teks `ON` ditampilkan sebagai berikut:

Keluaran Downlink Teks

```
Received text: ON
```

Jika data yang diterima mengandung byte yang tidak dapat dicetak, program menampilkan setiap byte dalam format heksadesimal:

Menampilkan Downlink Biner

```
Serial.print("Received hexadecimal data: ");

for (int i = 0; i < receiveStatus; i++) {
  if (static_cast<byte>( receivedData[i]) < 0x10) {
    Serial.print('0');
  }

  Serial.print(static_cast<byte>( receivedData[i]), HEX);
  Serial.print(' ');
}
```

Downlink biner dapat ditampilkan pada Serial Monitor sebagai berikut:

Keluaran Downlink Biner

```
Received hexadecimal data: 01 FF A0
```

4.6.2.4 Menerima Downlink

ANTARES dapat mengirimkan pesan downlink ke perangkat LoRaWAN. Karena program menggunakan Class A, downlink dikirimkan melalui receive window yang dibuka setelah uplink. Ketika downlink teks diterima, Serial Monitor menampilkan keluaran seperti berikut:

Keluaran Downlink Teks

```
Received text: ON
```

Ketika downlink berisi data biner yang tidak dapat dicetak, program menampilkannya dalam format heksadesimal:

Keluaran Downlink Biner

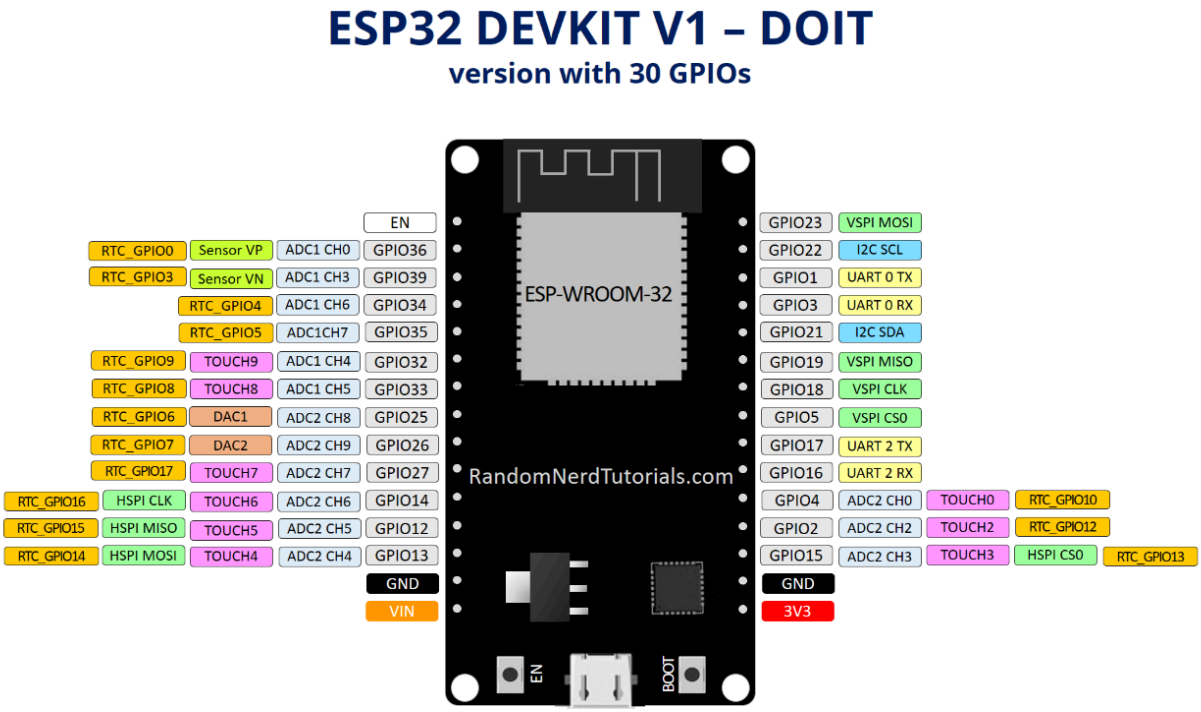
```
Received hexadecimal data: 01 FF A0
```

Jika perangkat tidak dapat mengirimkan data ke ANTARES, periksa Device Address, Network Session Key, Application Session Key, koneksi antena, jangkauan gateway, konfigurasi pin radio, kelas perangkat, metode aktivasi, dan konfigurasi kanal LoRaWAN.

LAMPIRAN

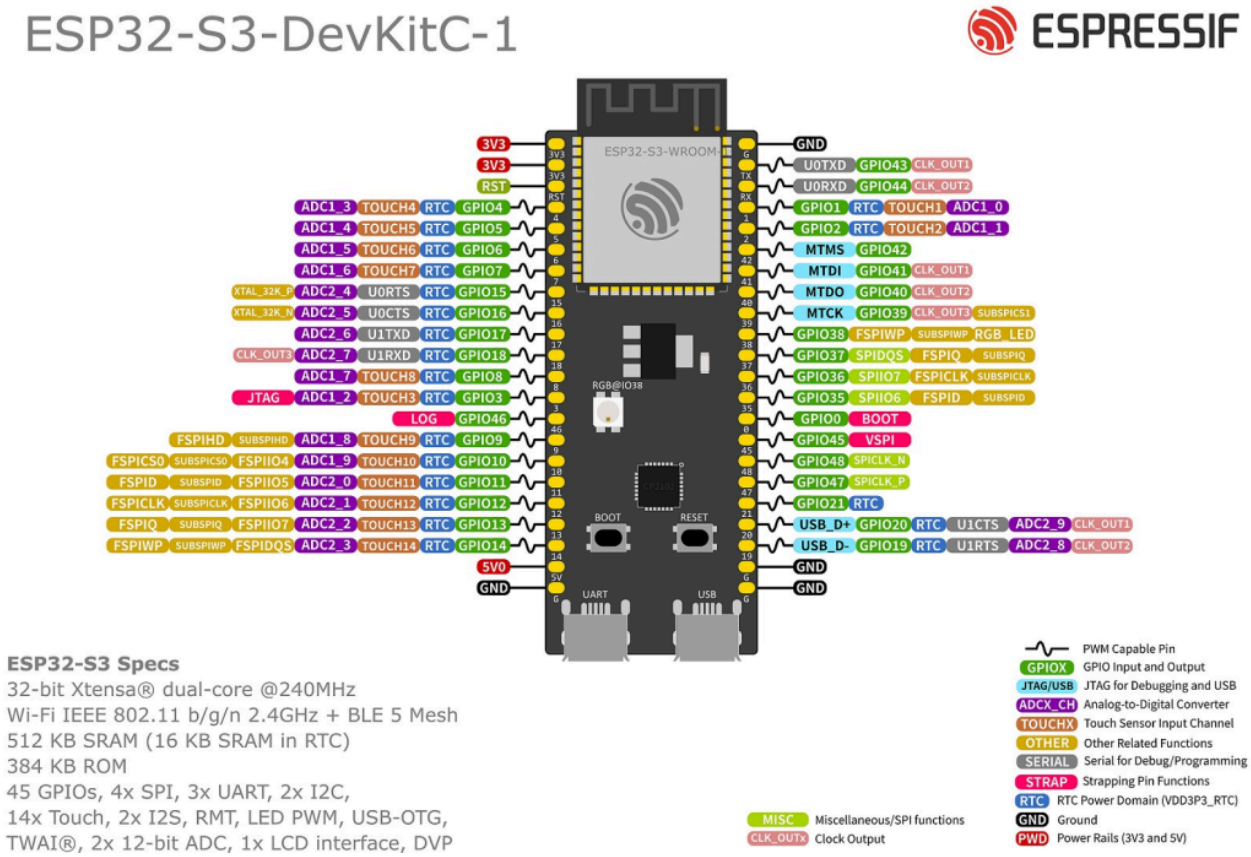
LAMPIRAN PINOUT DIAGRAM MIKROKONTROLER

ESP32 DEVKIT V1 - DOIT



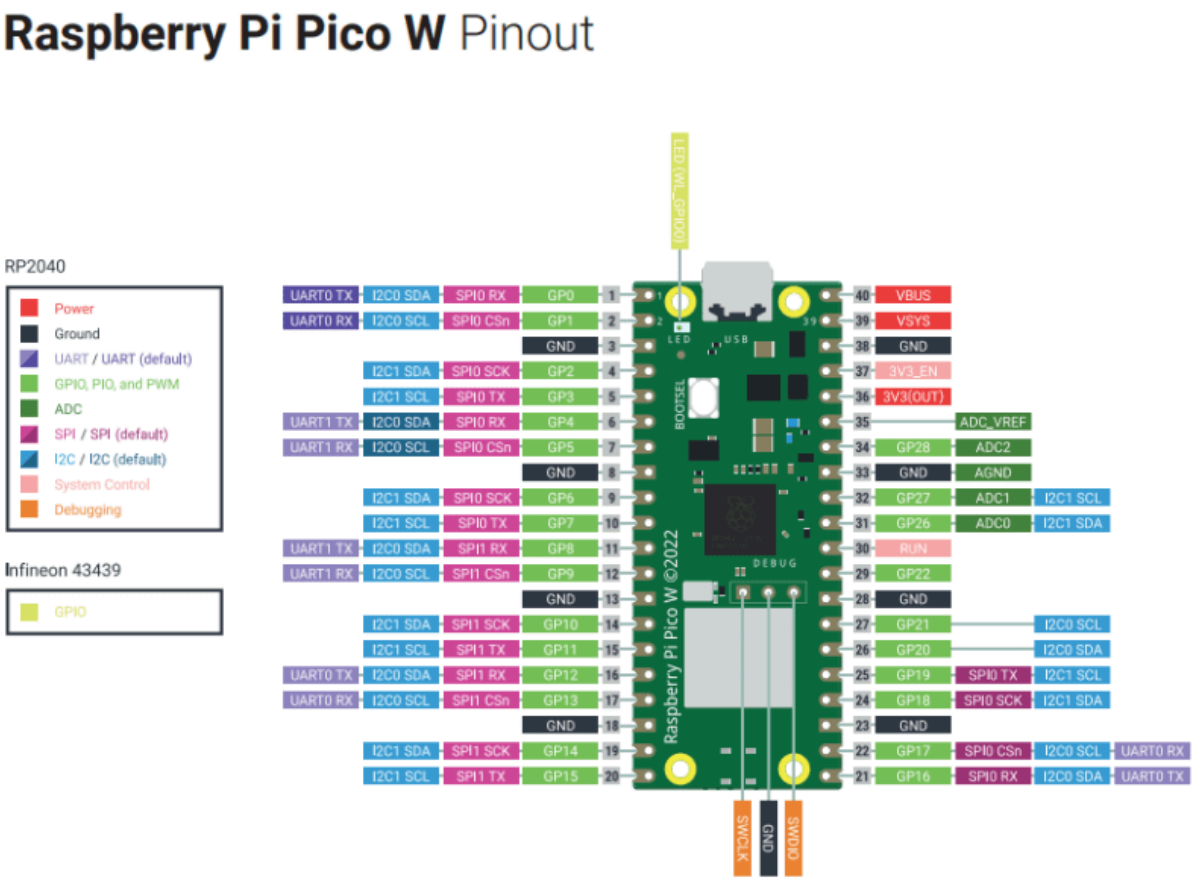
Lampiran 1: Pinout ESP32 DEVKIT V1 - DOIT

ESP32-S3



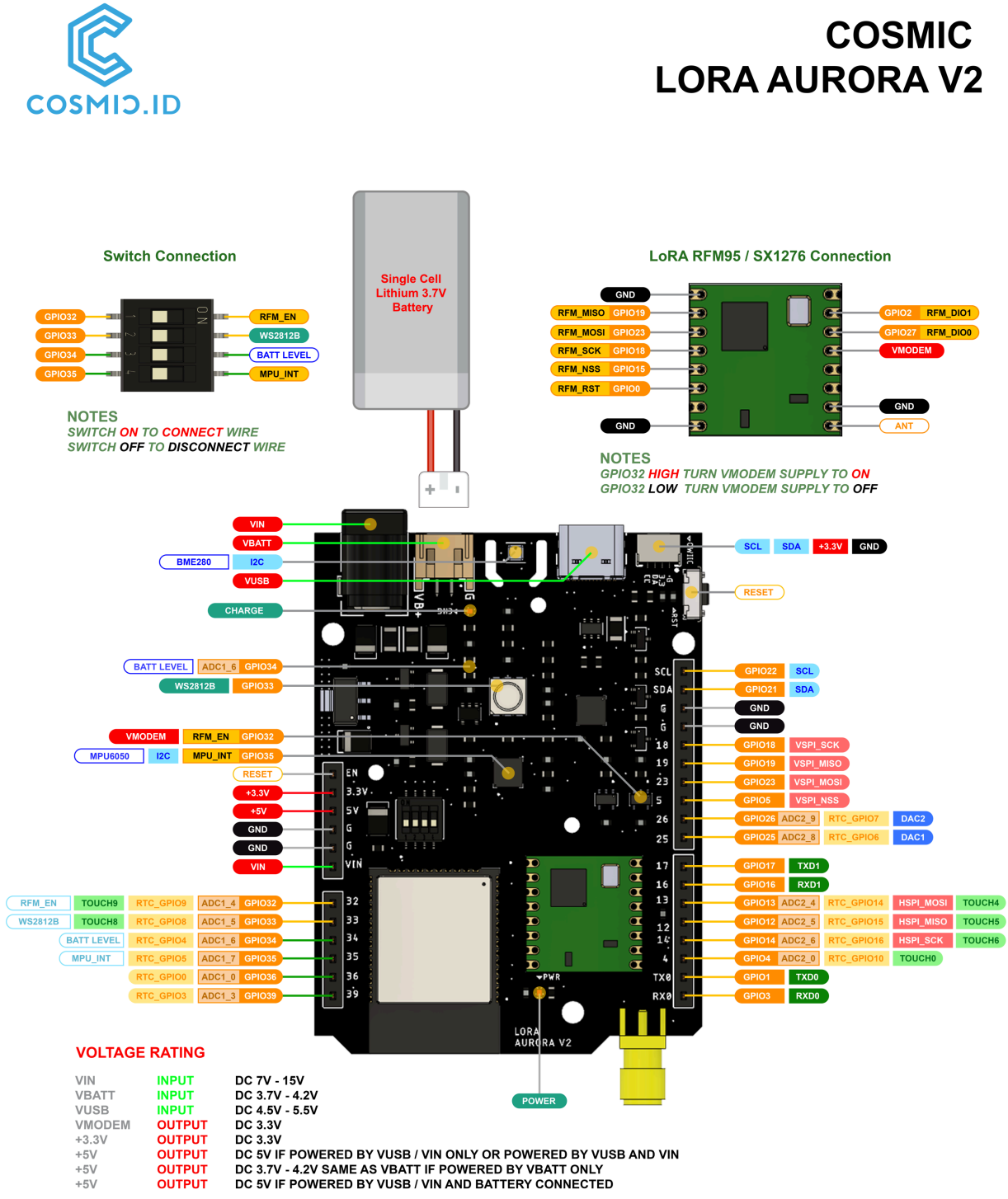
Lampiran 2: Pinout ESP32-S3

RASPBERRY PI PICO W



Lampiran 3: Pinout RASPBERRY PI PICO W

COSMIC LORA AURORA V2



Lampiran 4: Pinout Cosmic LoRa Aurora V2

LAMPIRAN MODUL 1

Source Code Bluetooth Master

Bluetooth Master

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String masterName = "ESP32-Master"; // Nama Master ESP
String slaveName = "ESP32-Slave";
bool connected = false;
int value = 0;

void setup() {
  Serial.begin(115200);

  SerialBT.begin(masterName, true); // Memulai Bluetooth
  Serial.println("The device started in master mode, make sure remote BT device is on!");

  connected = SerialBT.connect(slaveName); // Periksa koneksi Bluetooth
  if(connected) {
    Serial.println("Connected Successfully!");
  } else {
    while(!SerialBT.connected(10 * 1000)) { // Menunggu 10 detik
      Serial.println("Failed to connect!");
      Serial.println("Make sure remote device is available and in range, then restart");
    }
  }
  SerialBT.connect();
}

void loop() {
  if(value <= 10){
    SerialBT.println(value); // Mengirimkan data melalui Bluetooth
    Serial.println(value);
    value++;

    delay(1000);
  }
  else {
    value = value - 10;
  }
}
```

Source Code Bluetooth Slave

Bluetooth Slave

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

String slaveName = "ESP32-Slave"; // Nama Slave ESP
int value = 0;
String slaveValueStr;
int slaveValueInt;

void setup() {
  Serial.begin(115200);
  SerialBT.begin(slaveName); // Memulai Bluetooth sebagai perangkat penerima
  Serial.println("The device started, now you can pair it with bluetooth!");
}

void loop() {
  if(SerialBT.available()){
```

```
    slaveValueStr = SerialBT.readStringUntil('\n'); // Membaca data dari Master
    slaveValueInt = slaveValueStr.toInt();

    Serial.print("Data dari Master : ");
    Serial.println(slaveValueInt);
  }
}
```

Source Code BLE Central/Client

BLE Central/Client

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEClient.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

static BLEAddress *pServerAddress;   // Alamat device BLE Slave
static boolean doConnect = false;    // Flag untuk mulai koneksi
static boolean connected = false;    // Status koneksi
static BLERemoteCharacteristic* pRemoteCharacteristic; // Karakteristik remote
BLEClient*  pClient;                 // Objek client BLE

static void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify
) {

  //  Letakkan logika kontrol di sini

  String receivedData = "";
  for (size_t i = 0; i < length; i++) {
    receivedData += (char)pData[i];
  }

  Serial.print("Received from Slave: ");
  Serial.println(receivedData);
}

class MyClientCallbacks : public BLEClientCallbacks {
  void onConnect(BLEClient *pClient) override {
    connected = true;
    Serial.println("Connected to Peripheral!");
  }

  void onDisconnect(BLEClient *pClient) override {
    connected = false;
    Serial.println("Disconnected from Peripheral!");
  }
};

bool connectToServer(BLEAddress pAddress) {
  Serial.print("Connecting to Slave: ");
  Serial.println(pAddress.toString().c_str());

  // Buat client baru
  pClient = BLEDevice::createClient();
  pClient->setClientCallbacks(new MyClientCallbacks());

  // Coba koneksi ke alamat BLE Slave
  if (!pClient->connect(pAddress)) {
```

```
    Serial.println("Connection failed!");
    return false;
  }

  Serial.println("Connected to Slave!");

  // Ambil service dari Slave
  BLERemoteService* pRemoteService = pClient->getService(SERVICE_UUID);
  if (pRemoteService == nullptr) {
    Serial.println("Service not found!");
    pClient->disconnect();
    return false;
  }

  // Ambil characteristic dari service
  pRemoteCharacteristic = pRemoteService->getCharacteristic(CHARACTERISTIC_UUID);
  if (pRemoteCharacteristic == nullptr) {
    Serial.println("Characteristic not found!");
    pClient->disconnect();
    return false;
  }

  // Aktifkan notifikasi
  if (!pRemoteCharacteristic->canNotify()) {
    Serial.println("Characteristic does not support notifications!");
    pClient->disconnect();
    return false;
  }

  pRemoteCharacteristic->registerForNotify(notifyCallback);
  connected = true;

  Serial.println("Notification enabled!");
  return true;
}

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
 void onResult(BLEAdvertisedDevice advertisedDevice) override {
    Serial.print("Found device: ");
    Serial.println(advertisedDevice.toString().c_str());

    // Cek apakah device memiliki service UUID yang sesuai
    if (
      advertisedDevice.haveServiceUUID() &&
      advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))
    ) {
      Serial.println("Target Peripheral found!");
      BLEDevice::getScan()->stop();

      if (pServerAddress != nullptr) {
        delete pServerAddress;
      }

      pServerAddress = new BLEAddress(advertisedDevice.getAddress());
      doConnect = true;
    }
  }
};

void setup() {
  Serial.begin(115200);
  Serial.println("=== BLE Central/Client ===");

  // Inisialisasi BLE Master
  BLEDevice::init("ESP32_Central");

  // Mulai Scanning
```

```
  BLEScan *pBLEScan = BLEDevice::getScan();
  pBLEScan->setAdvertisedDeviceCallbacks(
    new MyAdvertisedDeviceCallbacks()
  );
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(80);
  pBLEScan->setActiveScan(true);

  Serial.println("Scanning for Peripheral...");
  pBLEScan->start(5, false); // Scan selama 5 detik
}

void loop() {
  // Jika menemukan device yang cocok, coba koneksi
  if (doConnect && pServerAddress != nullptr) {
    if (connectToServer(*pServerAddress)) {
      Serial.println("Ready to receive data!");
    } else {
      Serial.println("Connection failed. Scanning again...");
    }

    doConnect = false;
  }

  // Jika terputus, coba scan lagi
  if (!connected && !doConnect) {
    BLEDevice::getScan()->start(5, false);
  }

  delay(1000);
}
```

Source Code BLE Peripheral/Server

BLE Peripheral/Server

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID "d1f8ef04-1fa7-4806-ba3a-8115c0bcce6e"
#define CHARACTERISTIC_UUID "11d29a29-98ee-453e-8c87-2f95f8676519"

BLEServer *pServer = nullptr;
BLECharacteristic *pCharacteristic = nullptr;

bool deviceConnected = false;
bool oldDeviceConnected = false;

unsigned long previousSendTime = 0;
const unsigned long sendInterval = 1000;

class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) {
    deviceConnected = true;            // Set status terhubung
    Serial.println("Master connected!");
  }

  void onDisconnect(BLEServer *pServer) {
    deviceConnected = false;           // Set status terputus
    Serial.println("Master disconnected!");
  }
};

void BLE_init() {
  BLEDevice::init("ESP32_Peripheral");
```

```
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService =
    pServer->createService(BLEUUID(SERVICE_UUID));

  // Membuat characteristic (data) di dalam service
  // PROPERTY_READ     -> Client bisa baca data
  // PROPERTY_WRITE    -> Client bisa kirim data ke ESP32
  // PROPERTY_NOTIFY   -> ESP32 kirim data otomatis (tanpa ACK)
  // PROPERTY_INDICATE -> ESP32 kirim data otomatis (dengan ACK)
  // PROPERTY_WRITE_NR -> Client kirim data tanpa konfirmasi
  pCharacteristic = pService->createCharacteristic(
    BLEUUID(CHARACTERISTIC_UUID),
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
  );

  pCharacteristic->addDescriptor(new BLE2902());
  pCharacteristic->setValue("0");

  pService->start();

  BLEAdvertising *pAdvertising =
    BLEDevice::getAdvertising();

  pAdvertising->addServiceUUID(BLEUUID(SERVICE_UUID));
  pAdvertising->setScanResponse(true);

  BLEDevice::startAdvertising();

  Serial.println("Peripheral ready. Waiting for Central...");
}

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  BLE_init();
}

void loop() {
  if (
    deviceConnected &&
    millis() - previousSendTime >= sendInterval
  ) {
    previousSendTime = millis();

    String value = String(random(0, 101));

    // BLE hanya bisa mengirim jenis string C (null terminated "\0")
    pCharacteristic->setValue(value.c_str());
    pCharacteristic->notify();

    Serial.print("Sent: ");
    Serial.println(value);
  }

  if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();

    Serial.println("Advertising restarted...");
    oldDeviceConnected = false;
  }

```

```
99      if (deviceConnected && !oldDeviceConnected) {
100       oldDeviceConnected = true;
101     }
102   }
```

LAMPIRAN MODUL 2

WebSocket Python Server Source Code

WebSocket Python Server

```python
import asyncio # Supports asynchronous coroutine execution

# WebSocket library
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0" # Listen on all network interfaces
PORT: int = 8765

async def handle_client(websocket: ServerConnection) -> None:
  # Distinguish messages by the WebSocket path
  # The path used in this example is "/esp32"
  path: str = websocket.request.path

  # Select the path to process
  if path == "/esp32":
    print("ESP32 device connected")

    try:
      async for message in websocket:
        # Message from ESP32
        print( f"ESP32 message: {message}" )

        # Reply
        response: str = ( f"ACK from Raspberry Pi: {message}" )
        await websocket.send(response)
    except ConnectionClosed:
      print("ESP32 disconnected")

  # Add other paths as required
  # Default case for an unrecognized path
  else:
  print(f"Unknown path: {path}")

async def main() -> None:
  # Use the configuration defined above
  async with serve(handle_client, HOST, PORT):
  print(f"WebSocket server running " f"on {HOST}:{PORT}")

  # Wait for messages...
  await asyncio.Future()

if __name__ == "__main__":
  asyncio.run(main())
```

Source Code WebSocket dengan ESP32

WebSocket dengan ESP32

```cpp
#include <WiFi.h>
#include <WebSocketsClient.h>

const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char*    WS_HOST = "192.168.1.100";
const uint16_t WS_PORT = 8765;
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

```

```
bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000;
uint32_t counter = 0;

void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
  switch (type) {
    case WStype_CONNECTED:
      webSocketConnected = true;
      Serial.println("WebSocket connected.");
      webSocket.sendTXT("{\"status\":\"ESP32 connected\"}");
      break;

    case WStype_DISCONNECTED:
      webSocketConnected = false;
      Serial.println("WebSocket disconnected.");
      break;

    case WStype_TEXT:
      Serial.print("Message from server: ");
      Serial.write(payload, length);
      Serial.println();
      break;

    case WStype_ERROR:
      Serial.println("WebSocket error.");
      break;

    default:
      break;
  }
}

void connectToWiFi() {
  Serial.print("Connecting to Wi-Fi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected.");
  Serial.print("ESP32 IP address: ");
  Serial.println(WiFi.localIP());
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  connectToWiFi();

  webSocket.begin(WS_HOST, WS_PORT, WS_PATH);
  webSocket.onEvent(webSocketEvent);
  webSocket.setReconnectInterval(5000);
  WiFi.setAutoReconnect(true);
}

void loop() {
  webSocket.loop();

  unsigned long currentTime = millis();
  if (
    webSocketConnected &&
```

```
      currentTime - previousSendTime >= SEND_INTERVAL
    ) {
      previousSendTime = currentTime;

      String message = "{\"counter\":";
      message += counter;
      message += "}";

      webSocket.sendTXT(message);
      Serial.print("Sent: ");
      Serial.println(message);
      counter++;
    }
  }
```

LAMPIRAN MODUL 3

Source Code untuk Publish MQTT ke Mosquitto MQTT

Publish MQTT ke Mosquitto MQTT

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, Adafruit IO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-publish
mqtt_publish_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "uus9asndajsndiqiwd10wid01n1kwd"

# Angka yang akan dipublish
counter = 0

# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

mqtt_client.connect()

try:
  while True:
    # Generate some dummy data that changes every loop
    counter += 3

    # Get the current timestamp
    current_time = utime.localtime()
    timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
      current_time[0], current_time[1], current_time[2],
      current_time[3], current_time[4], current_time[5]
    )

    # Publish the data to the topic!
```

```
    print(f'{timestamp} - Publish {counter}')
    mqtt_client.publish(mqtt_publish_topic, str(counter))

    # Delay a bit to avoid hitting the rate limit
    time.sleep(3)

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

Source Code untuk Subscribe MQTT dari Mosquitto MQTT

Subscribe MQTT dari Mosquitto MQTT

```
import network
import time
from umqtt.simple import MQTTClient
import utime

# Isi nama Wi-Fi dan password yang terkoneksi bersama broker lokal di sini
wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"

# Isi sesuai dengan broker yang akan dipakai (Mosquitto, HiveMQ, Adafruit IO)
# dan lain-lain
mqtt_host     = "<MQTT HOST ADDRESS>"

# (Opsional) Bila menggunakan autentikasi
mqtt_username = "<USERNAME FROM MOSQUITTO MQTT>"
mqtt_password = "<PASSWORD FROM MOSQUITTO MQTT>"

# Isi sesuai dengan topik yang ingin di-subscribe
mqtt_receive_topic = "<YOUR mqtt_topic/feed_director>"

# Isi ID client random (harus unik tiap client)
mqtt_client_id = "iij0dfq8f012nif2jnfijn"

# Koneksi ke WiFi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(wifi_ssid, wifi_password)

while wlan.isconnected() == False:
  print('Waiting for connection...')
  time.sleep(1)

print("Connected to WiFi")

# Initialize our MQTTClient and connect to the MQTT server
mqtt_client = MQTTClient(
  client_id=mqtt_client_id,
  server=mqtt_host,
  # Bila menggunakan autentikasi
  user=mqtt_username,
  password=mqtt_password
)

def mqtt_subscription_callback(topic, message):
  # Get the current timestamp
  current_time = utime.localtime()

  timestamp = "{:04}-{:02}-{:02} {:02}:{:02}:{:02}".format(
    current_time[0], current_time[1], current_time[2],
```

```
      current_time[3], current_time[4], current_time[5]
    )
    # Debug print out of what was received over MQTT
    print (f'{timestamp} - Topic {topic} received message {message}')

# Before connecting, tell the MQTT client to use the callback
mqtt_client.set_callback(mqtt_subscription_callback)
mqtt_client.connect()

mqtt_client.subscribe(mqtt_receive_topic)
print("Connected and subscribed")

try:
  while True:
    # Infinitely wait for messages on the topic.
    # Note wait_msg() is a blocking call, if you're doing multiple things
    # on the Pico you may want to look at putting this on another thread.
    print(f'Waiting for messages on {mqtt_receive_topic}')
    mqtt_client.wait_msg()

# Jika ada masalah pada saat pengiriman
except Exception as e:
  print(f'MQTT error: {e}')

# Ketika program python dimatikan
finally:
  mqtt_client.disconnect()
```

LAMPIRAN MODUL 4

Source Code untuk Transmitter LoRa

Transmitter LoRa

```
// Menggunakan dua library, yaitu SPI.h dan LoRa.h oleh Sandeep Mistry
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Ganti sync word (0xF3) menyesuaikan transceiver
  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  Serial.print("Sending packet: ");
  Serial.println(counter);

  //Send LoRa packet to receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
  LoRa.print(counter);
  LoRa.endPacket();

  counter++;
  delay(5000);
}
```

Source Code untuk Receiver LoRa

Receiver LoRa

```
// Menggunakan dua library, yaitu SPI.h dan LoRa.h oleh Sandeep Mistry
#include <SPI.h>
#include <LoRa.h>

// Tentukan pin yang digunakan oleh modul LoRa
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Inisialisasi mode pin LoRa
  digitalWrite(LORA_EN, HIGH);  // LoRa chip dinyalakan

  // Inisialisasi Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Setup pin modul LoRa
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Ganti LoRa.begin(---E-) argument dengan frekuensi yang sesuai regulasi wilayah
  // 433E6 untuk region Asia
  // 866E6 untuk region Europe
  // 915E6 untuk region North America
  // 920E6 untuk Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Sync word membantu menyaring paket dengan nilai berbeda
  // Memiliki rentang 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  // Mencoba untuk membuka paket yang diterima
  int packetSize = LoRa.parsePacket();

  // Menerima paket
  if (packetSize) {
    Serial.print("Received packet '");

    // Membaca paket yang diterima
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Menampilkan RSSI/Sinyal dari paket yang diterima (dB)
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

LoRaWAN Source Code

LoRaWAN Microcontroller Source Code

```
#include <SPI.h>
#include <lorawan.h>

#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18

const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";

const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;

const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
  .DIO0 = LORA_DIO0,
  .DIO1 = LORA_DIO1,
};

void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz.
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);

  // It will be available to multiple LoRaWAN channels.
```

```
  lora.setChannel(MULTI);

  // The transmission power should comply with the applicable regional frequency
  // regulations and the requirements of the LoRaWAN network.
  lora.setTxPower1(15);

  // ABP credentials.
  // Because ABP uses predefined session parameters,
  // the device does not perform an OTAA join procedure before transmitting data.
  lora.setNwkSKey(nwkSKey);
  lora.setAppSKey(appSKey);
  lora.setDevAddr(devAddr);

  Serial.println("LoRaWAN Class A ABP configuration completed");
}

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);

    Serial.print("Sending: ");
    Serial.println(message);

    lora.sendUplink(message, strlen(message), 0, 5);

    messageCounter++;

    channel = lora.getChannel();

    Serial.print("Channel: ");
    Serial.println(channel);
  }

  lora.update();

  receiveStatus = lora.readData(receivedData);

  if (receiveStatus > 0) {
    bool isText = true;

    for (int i = 0; i < receiveStatus; i++) {
      if (!(
        (receivedData[i] >= 32 && receivedData[i] <= 126) ||
        receivedData[i] == 10 ||
        receivedData[i] == 13
      )) {
        isText = false;
        break;
      }
    }

    if (isText) {
      Serial.print("Received text: ");

      for (int i = 0; i < receiveStatus; i++) {
        Serial.print(receivedData[i]);
      }
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>(receivedData[i]) < 0x10) {
          Serial.print('0');
```

```
132           }
133
134           Serial.print(static_cast<byte>(receivedData[i]), HEX);
135           Serial.print(' ');
136         }
137       }
138
139       Serial.println();
140     }
141   }
```